

Bitcoin and Cryptocurrency Technologies

**Arvind Narayanan, Joseph Bonneau, Edward Felten,
Andrew Miller, Steven Goldfeder**

with a preface by Jeremy Clark

Draft — Feb 9, 2016

Feedback welcome! Email bitcoinbook@lists.cs.princeton.edu

For the latest draft and supplementary materials including programming assignments,
see our [Coursera course](#).

The official version of this book will be published by Princeton University Press in 2016.
If you'd like to be notified when it's available, please sign up [here](#).

Chapter 1: Introduction to Cryptography & Cryptocurrencies

All currencies need some way to control supply and enforce various security properties to prevent cheating. In fiat currencies, organizations like central banks control the money supply and add anti-counterfeiting features to physical currency. These security features raise the bar for an attacker, but they don't make money impossible to counterfeit. Ultimately, law enforcement is necessary for stopping people from breaking the rules of the system.

Cryptocurrencies too must have security measures that prevent people from tampering with the state of the system, and from equivocating, that is, making mutually inconsistent statements to different people. If Alice convinces Bob that she paid him a digital coin, for example, she should not be able to convince Carol that she paid her that same coin. But unlike fiat currencies, the security rules of cryptocurrencies need to be enforced purely technologically and without relying on a central authority.

As the word suggests, cryptocurrencies make heavy use of cryptography. Cryptography provides a mechanism for securely encoding the rules of a cryptocurrency system in the system itself. We can use it to prevent tampering and equivocation, as well as to encode the rules for creation of new units of the currency into a mathematical protocol. Before we can properly understand cryptocurrencies then, we'll need to delve into the cryptographic foundations that they rely upon.

Cryptography is a deep academic research field utilizing many advanced mathematical techniques that are notoriously subtle and complicated to understand. Fortunately, Bitcoin only relies on a handful of relatively simple and well-known cryptographic constructions. In this chapter, we'll specifically study cryptographic hashes and digital signatures, two primitives that prove to be very useful for building cryptocurrencies. Future chapters will introduce more complicated cryptographic schemes, such as zero-knowledge proofs, that are used in proposed extensions and modifications to Bitcoin.

Once we've learnt the necessary cryptographic primitives, we'll discuss some of the ways in which those are used to build cryptocurrencies. We'll complete this chapter with some examples of simple cryptocurrencies that illustrate some of the design challenges that we need to deal with.

1.1 Cryptographic Hash Functions

The first cryptographic primitive that we'll need to understand is a ***cryptographic hash function***. A ***hash function*** is a mathematical function with the following three properties:

- Its input can be any string of any size.
- It produces a fixed size output. For the purpose of making the discussion in this chapter concrete, we will assume a 256-bit output size. However, our discussion holds true for any output size as long as it is sufficiently large.
- It is efficiently computable. Intuitively this means that for a given input string, you can figure

out what the output of the hash function is in a reasonable amount of time. More technically, computing the hash of an n -bit string should have a running time that is $O(n)$.

Those properties define a general hash function, one that could be used to build a data structure such as a hash table. We're going to focus exclusively on *cryptographic* hash functions. For a hash function to be cryptographically secure, we're going to require that it has the following three additional properties: (1) collision-resistance, (2) hiding, (3) puzzle-friendliness.

We'll look more closely at each of these properties to gain an understanding of why it's useful to have a function that behaves that way. The reader who has studied cryptography should be aware that the treatment of hash functions in this book is a bit different from a standard cryptography textbook. The puzzle-friendliness property, in particular, is not a general requirement for cryptographic hash functions, but one that will be useful for cryptocurrencies specifically.

Property 1: Collision-resistance. The first property that we need from a cryptographic hash function is that it's collision-resistant. A collision occurs when two distinct inputs produce the same output. A hash function $H(\cdot)$ is collision-resistant if nobody can find a collision. Formally:

Collision-resistance: A hash function H is said to be collision resistant if it is infeasible to find two values, x and y , such that $x \neq y$, yet $H(x)=H(y)$.

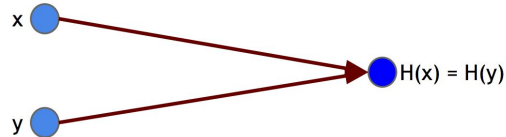


Figure 1.1 A hash collision. x and y are distinct values, yet when input into hash function H , they produce the same output.

Notice that we said *nobody can find* a collision, but we did not say that no collisions exist. Actually, we know for a fact that collisions do exist, and we can prove this by a simple counting argument. The input space to the hash function contains all strings of all lengths, yet the output space contains only strings of a specific fixed length. Because the input space is larger than the output space (indeed, the input space is infinite, while the output space is finite), there must be input strings that map to the same output string. In fact, by the Pigeonhole Principle there will necessarily be a very large number of possible inputs that map to any particular output.

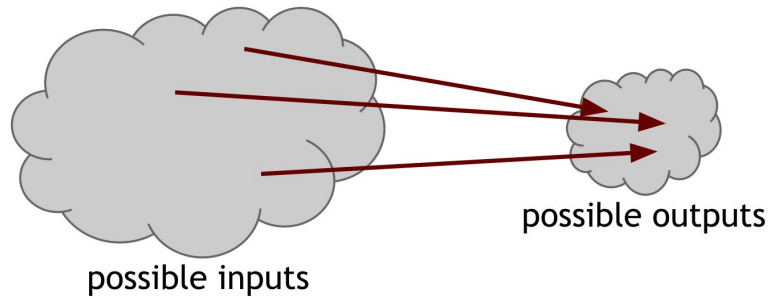


Figure 1.2 Because the number of inputs exceeds the number of outputs, we are guaranteed that there must be at least one output to which the hash function maps more than one input.

Now, to make things even worse, we said that it has to be impossible to find a collision. Yet, there are methods that are guaranteed to find a collision. Consider the following simple method for finding a collision for a hash function with a 256-bit output size: pick $2^{256} + 1$ distinct values, compute the hashes of each of them, and check if there are any two outputs that are equal. Since we picked more inputs than possible outputs, some pair of them must collide when you apply the hash function.

The method above is guaranteed to find a collision. But if we pick random inputs and compute the hash values, we'll find a collision with high probability long before examining $2^{256} + 1$ inputs. In fact, if we randomly choose just $2^{130} + 1$ inputs, it turns out there's a 99.8% chance that at least two of them are going to collide. The fact that we can find a collision by only examining roughly the square root of the number of possible outputs results from a phenomenon in probability known as the **birthday paradox**. In the homework questions at the end of this chapter, we will examine this in more detail.

This collision-detection algorithm works for every hash function. But, of course, the problem with it is that this takes a very, very long time to do. For a hash function with a 256-bit output, you would have to compute the hash function $2^{256} + 1$ times in the worst case, and about 2^{128} times on average. That's of course an astronomically large number — if a computer calculates 10,000 hashes per second, it would take more than one octillion (10^{27}) years to calculate 2^{128} hashes! For another way of thinking about this, we can say that, if every computer ever made by humanity was computing since the beginning of the entire universe, up to now, the odds that they would have found a collision is still infinitesimally small. So small that it's way less than the odds that the Earth will be destroyed by a giant meteor in the next two seconds.

We have thus seen a general but impractical algorithm to find a collision for *any* hash function. A more difficult question is: is there some other method that could be used on a particular hash function in order to find a collision? In other words, although the generic collision detection algorithm is not feasible to use, there still may be some other algorithm that can efficiently find a collision for a specific hash function.

Consider, for example, the following hash function:

$$H(x) = x \bmod 2^{256}$$

This function meets our requirements of a hash function as it accepts inputs of any length, returns a fixed sized output (256 bits), and is efficiently computable. But this function also has an efficient method for finding a collision. Notice that this function just returns the last 256 bits of the input. One collision then would be the values 3 and $3 + 2^{256}$. This simple example illustrates that even though our generic collision detection method is not usable in practice, there are at least some hash functions for which an efficient collision detection method does exist.

Yet for other hash functions, we don't know if such methods exist. We suspect that they are collision resistant. However, there are no hash functions *proven* to be collision-resistant. The cryptographic hash functions that we rely on in practice are just functions for which people have tried really, really hard to find collisions and haven't yet succeeded. In some cases, such as the old MD5 hash function, collisions were eventually found after years of work, leading the function to be deprecated and phased out of practical use. And so we choose to believe that those are collision resistant.

Application: Message digests Now that we know what collision-resistance is, the logical question is: What is collision-resistance useful for? Here's one application: If we know that two inputs x and y to a collision-resistant hash function H are different, then it's safe to assume that their hashes $H(x)$ and $H(y)$ are different — if someone knew an x and y that were different but had the same hash, that would violate our assumption that H is collision resistant.

This argument allows us to use hash outputs as a **message digest**. Consider SecureBox, an authenticated online file storage system that allows users to upload files and ensure their integrity when they download them. Suppose that Alice uploads really large file, and wants to be able to verify later that the file she downloads is the same as the one she uploads. One way to do that would be to save the whole big file locally, and directly compare it to the file she downloads. While this works, it largely defeats the purpose of uploading it in the first place; if Alice needs to have access to a local copy of the file to ensure its integrity, she can just use the local copy directly.

Collision-free hashes provide an elegant and efficient solution to this problem. Alice just needs to remember the hash of the original file. When she later downloads the file from SecureBox, she computes the hash of the downloaded file and compares it to the one she stored. If the hashes are the same, then she can conclude that the file is indeed the one she uploaded, but if they are different, then Alice can conclude that the file has been tampered with. Remembering the hash thus allows her to detect *accidental* corruption of the file during transmission or on SecureBox's servers, but also *intentional* modification of the file by the server. Such guarantees in the face of potentially malicious behavior by other entities are at the core of what cryptography gives us.

The hash serves as a fixed length digest, or unambiguous summary, of a message. This gives us a very efficient way to remember things we've seen before and recognize them again. Whereas the entire file might have been gigabytes long, the hash is of fixed length, 256-bits for the hash function in our example. This greatly reduces our storage requirement. Later in this chapter and throughout the book, we'll see applications for which it's useful to use a hash as a message digest.

Property 2: Hiding The second property that we want from our hash functions is that it's **hiding**. The hiding property asserts that if we're given the output of the hash function $y = H(x)$, there's no feasible way to figure out what the input, x , was. The problem is that this property can't be true in the stated form. Consider the following simple example: we're going to do an experiment where we flip a coin. If the result of the coin flip was heads, we're going to announce the hash of the string "heads". If the result was tails, we're going to announce the hash of the string "tails".

We then ask someone, an adversary, who didn't see the coin flip, but only saw this hash output, to figure out what the string was that was hashed (we'll soon see why we might want to play games like this). In response, they would simply compute both the hash of the string "heads" and the hash of the string "tails", and they could see which one they were given. And so, in just a couple steps, they can figure out what the input was.

The adversary was able to guess what the string was because there were only two possible values of x , and it was easy for the adversary to just try both of them. In order to be able to achieve the hiding property, it needs to be the case that there's no value of x which is particularly likely. That is, x has to be chosen from a set that's, in some sense, very spread out. If x is chosen from such a set, this method of trying a few values of x that are especially likely will not work.

The big question is: can we achieve the hiding property when the values that we want do not come from a spread-out set as in our "heads" and "tails" experiment? Fortunately, the answer is yes! So perhaps we can hide even an input that's not spread out by concatenating it with another input that *is* spread out. We can now be slightly more precise about what we mean by hiding (the double vertical bar $\parallel$ denotes concatenation).

Hiding. A hash function H is hiding if: when a secret value r is chosen from a probability distribution that has *high min-entropy*, then given $H(r \parallel x)$ it is infeasible to find x .

In information-theory, **min-entropy** is a measure of how predictable an outcome is, and high min-entropy captures the intuitive idea that the distribution (i.e., random variable) is very spread out. What that means specifically is that when we sample from the distribution, there's no particular value that's likely to occur. So, for a concrete example, if r is chosen uniformly from among all of the strings that are 256 bits long, then any particular string was chosen with probability $1/2^{256}$, which is an infinitesimally small value.

Application: Commitments. Now let's look at an application of the hiding property. In particular, what we want to do is something called a **commitment**. A commitment is the digital analog of taking a value, sealing it in an envelope, and putting that envelope out on the table where everyone can see it. When you do that, you've committed yourself to what's inside the envelope. But you haven't opened it, so even though you've committed to a value, the value remains a secret from everyone else. Later, you can open the envelope and reveal the value that you committed to earlier.

Commitment scheme. A commitment scheme consists of two algorithms:

- **com** := **commit**(*msg*, *nonce*) The commit function takes a message and secret random value, called a nonce, as input and returns a commitment.
- **verify**(*com*, *msg*, *nonce*) The verify function takes a commitment, nonce, and message as input. It returns true if $com == \text{commit}(msg, nonce)$ and false otherwise.

We require that the following two security properties hold:

- **Hiding:** Given *com*, it is infeasible to find *msg*
- **Binding:** It is infeasible to find two pairs (*msg*, *nonce*) and (*msg'*, *nonce'*) such that $msg \neq msg'$ and $\text{commit}(msg, nonce) == \text{commit}(msg', nonce')$

To use a commitment scheme, we first need to generate a random *nonce*. We then apply the *commit* function to this nonce together with *msg*, the value being committed to, and we publish the commitment *com*. This stage is analogous to putting the sealed envelope on the table. At a later point, if we want to reveal the value that they committed to earlier, we publish the random nonce that we used to create this commitment, and the message, *msg*. Now, anybody can verify that *msg* was indeed the message committed to earlier. This stage is analogous to opening up the envelope.

Every time you commit to a value, it is important that you choose a new random value *nonce*. In cryptography, the term *nonce* is used to refer to a value that can only be used once.

The two security properties dictate that the algorithms actually behave like sealing and opening an envelope. First, given *com*, the commitment, someone looking at the envelope can't figure out what the message is. The second property is that it's binding. This ensures that when you commit to what's in the envelope, you can't change your mind later. That is, it's infeasible to find two different messages, such that you can commit to one message, and then later claim that you committed to another.

So how do we know that these two properties hold? Before we can answer this, we need to discuss how we're going to actually implement a commitment scheme. We can do so using a cryptographic hash function. Consider the following commitment scheme:

$\text{commit}(msg, nonce) := H(nonce \parallel msg)$ where *nonce* is a random 256-bit value

To commit to a message, we generate a random 256-bit nonce. Then we concatenate the nonce and the message and return the hash of this concatenated value as the commitment. To verify, someone will compute this same hash of the nonce they were given concatenated with the message. And they will check whether that's equal to the commitment that they saw.

Take another look at the two properties that we require of our commitment schemes. If we substitute the instantiation of *commit* and *verify* as well as $H(nonce \parallel msg)$ for *com*, then these properties

become:

- **Hiding:** Given $H(\text{nonce} \parallel \text{msg})$, it is infeasible to find msg
- **Binding:** It is infeasible to find two pairs $(\text{msg}, \text{nonce})$ and $(\text{msg}', \text{nonce}')$ such that $\text{msg} \neq \text{msg}'$ and $H(\text{nonce} \parallel \text{msg}) = H(\text{nonce}' \parallel \text{msg}')$

The *hiding* property of commitments is exactly the hiding property that we required for our hash functions. If *key* was chosen as a random 256-bit value then the hiding property says that if we hash the concatenation of *key* and the message, then it's infeasible to recover the message from the hash output. And it turns out that the *binding property* is implied by¹ the collision-resistant property of the underlying hash function. If the hash function is collision-resistant, then it will be infeasible to find distinct values *msg* and *msg'* such that $H(\text{nonce} \parallel \text{msg}) = H(\text{nonce}' \parallel \text{msg}')$ since such values would indeed be a collision.

Therefore, if *H* is a hash function that is collision-resistant and hiding, this commitment scheme will work, in the sense that it will have the necessary security properties.

Property 3: Puzzle friendliness. The third security property we're going to need from hash functions is that they are puzzle-friendly. This property is a bit complicated. We will first explain what the technical requirements of this property are and then give an application that illustrates why this property is useful.

Puzzle friendliness. A hash function *H* is said to be puzzle-friendly if for every possible *n*-bit output value *y*, if *k* is chosen from a distribution with high min-entropy, then it is infeasible to find *x* such that $H(k \parallel x) = y$ in time significantly less than 2^n .

Intuitively, what this means is that if someone wants to target the hash function to come out to some particular output value *y*, that if there's part of the input that is chosen in a suitably randomized way, it's very difficult to find another value that hits exactly that target.

Application: Search puzzle. Now, let's consider an application that illustrates the usefulness of this property. In this application, we're going to build a **search puzzle**, a mathematical problem which requires searching a very large space in order to find the solution. In particular, a search puzzle has no shortcuts. That is, there's no way to find a valid solution other than searching that large space.

¹ The reverse implications do not hold. That is, it's possible that you can find collisions, but none of them are of the form $H(\text{nonce} \parallel \text{msg}) = H(\text{nonce}' \parallel \text{msg}')$. For example, if you can only find a collision in which two distinct nonces generate the same commitment for the same message, then the commitment scheme is still binding, but the underlying hash function is not collision-resistant.

Search puzzle. A search puzzle consists of

- a hash function, H ,
- a value, id (which we call the **puzzle-ID**), chosen from a high min-entropy distribution
- and a target set Y

A solution to this puzzle is a value, x , such that

$$H(id \parallel x) \in Y.$$

The intuition is this: if H has an n -bit output, then it can take any of 2^n values. Solving the puzzle requires finding an input so that the output falls within the set Y , which is typically much smaller than the set of all outputs. The size of Y determines how hard the puzzle is. If Y is the set of all n -bit strings the puzzle is trivial, whereas if Y has only 1 element the puzzle is maximally hard. The fact that the puzzle id has high min-entropy ensures that there are no shortcuts. On the contrary, if a particular value of the ID were likely, then someone could cheat, say by pre-computing a solution to the puzzle with that ID .

If a search puzzle is puzzle-friendly, this implies that there's no solving strategy for this puzzle which is much better than just trying random values of x . And so, if we want to pose a puzzle that's difficult to solve, we can do it this way as long as we can generate puzzle-IDs in a suitably random way. We're going to use this idea later when we talk about Bitcoin mining, which is a sort of computational puzzle.

SHA-256. We've discussed three properties of hash functions, and one application of each of those. Now let's discuss a particular hash function that we're going to use a lot in this book. There are lots of hash functions in existence, but this is the one Bitcoin uses primarily, and it's a pretty good one to use. It's called **SHA-256**.

Recall that we require that our hash functions work on inputs of arbitrary length. Luckily, as long as we can build a hash function that works on fixed-length inputs, there's a generic method to convert it into a hash function that works on arbitrary-length inputs. It's called the **Merkle-Damgard transform**. SHA-256 is one of a number of commonly used hash functions that make use of this method. In common terminology, the underlying fixed-length collision-resistant hash function is called the **compression function**. It has been proven that if the underlying compression function is collision resistant, then the overall hash function is collision resistant as well.

The Merkle-Damgard transform is quite simple. Say the compression function takes inputs of length m and produces an output of a smaller length n . The input to the hash function, which can be of any size, is divided into **blocks** of length $m-n$. The construction works as follows: pass each block together with the output of the previous block into the compression function. Notice that input length will then be $(m-n) + n = m$, which is the input length to the compression function. For the first block, to which there is no previous block output, we instead use an **Initialization Vector (IV)**. This number is reused for every call to the hash function, and in practice you can just look it up in a standards document. The last block's output is the result that you return.

SHA-256 uses a compression function that takes 768-bit input and produces 256-bit outputs. The block size is 512 bits. See Figure 1.3 for a graphical depiction of how SHA-256 works.

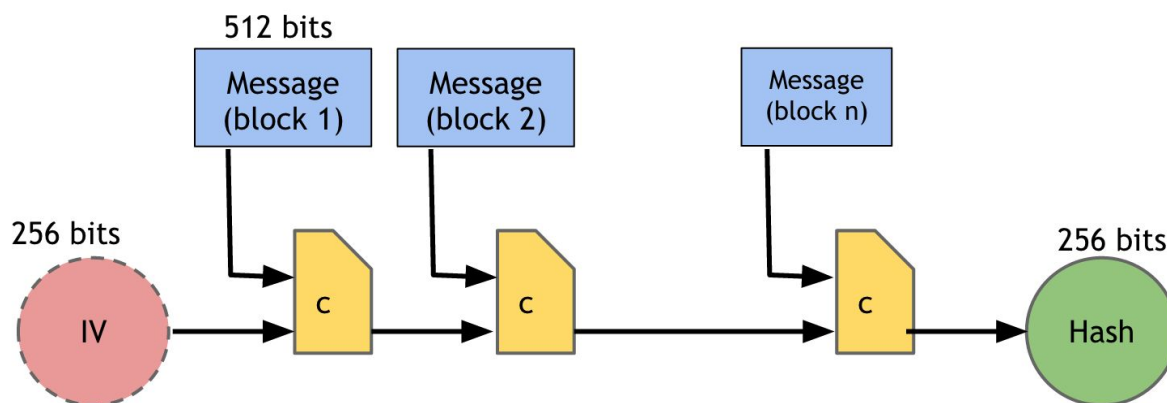


Figure 1.3: SHA-256 Hash Function (simplified). SHA-256 uses the Merkle-Damgard transform to turn a fixed-length collision-resistant compression function into a hash function that accepts arbitrary-length inputs. The input is “padded” so that its length is a multiple of 512 bits.

We’ve talked about hash functions, cryptographic hash functions with special properties, applications of those properties, and a specific hash function that we use in Bitcoin. In the next section, we’ll discuss ways of using hash functions to build more complicated data structures that are used in distributed systems like Bitcoin.

Sidebar: modeling hash functions. Hash functions are the Swiss Army knife of cryptography: they find a place in a spectacular variety of applications. The flip side to this versatility is that different applications require slightly different properties of hash functions to ensure security. It’s proven notoriously hard to pin down a list of hash function properties that would result in provable security across the board.

In this text, we’ve selected three properties that are crucial to the way that hash functions are used in Bitcoin and other cryptocurrencies. Even within this space, not all of these properties are necessary for every use of hash functions. For example, puzzle-friendliness is only important in Bitcoin mining, as we’ll see.

Designers of secure systems often throw in the towel and model hash functions as functions that output an independent random value for every possible input. The use of this “random oracle model” for proving security remains controversial in cryptography. Regardless of one’s position on this debate, reasoning about how to reduce the security properties that we want in our applications to fundamental properties of the underlying primitives is a valuable intellectual exercise for building secure systems. Our presentation in this chapter is designed to help you learn this skill.

1.2 Hash Pointers and Data Structures

In this section, we're going to discuss **hash pointers** and their applications. A hash pointer is a data structure that turns out to be useful in many of the systems that we will talk about. A hash pointer is simply a pointer to where some information is stored together with a cryptographic hash of the information. Whereas a regular pointer gives you a way to retrieve the information, a hash pointer also gives you a way to verify that the information hasn't changed.

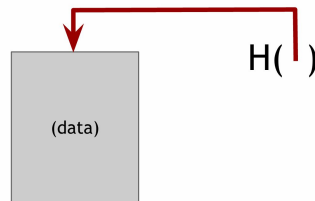


Figure 1.4 Hash pointer. A hash pointer is a pointer to where data is stored together with a cryptographic hash of the value of that data at some fixed point in time.

We can use hash pointers to build all kinds of data structures. Intuitively, we can take a familiar data structure that uses pointers such as a linked list or a binary search tree and implement it with hash pointers, instead of pointers as we normally would.

Block chain. In Figure 1.5, we built a linked list using hash pointers. We're going to call this data structure a **block chain**. Whereas as in a regular linked list where you have a series of blocks, each block has data as well as a pointer to the previous block in the list, in a block chain the previous block pointer will be replaced with a hash pointer. So each block not only tells us where the value of the previous block was, but it also contains a digest of that value that allows us to verify that the value hasn't changed. We store the head of the list, which is just a regular hash-pointer that points to the most recent data block.

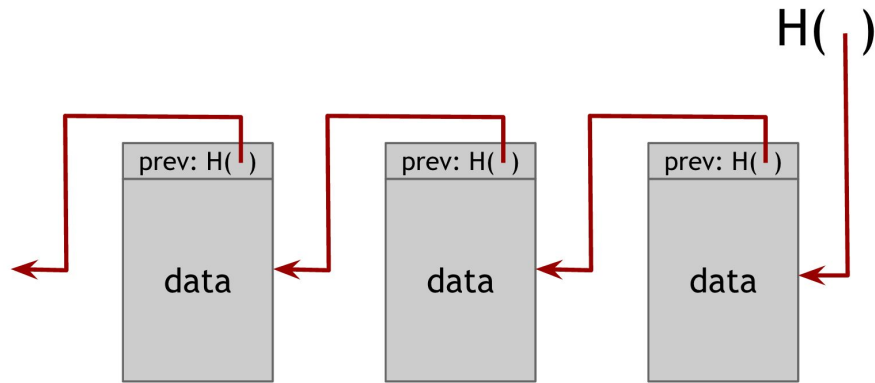


Figure 1.5 Block chain. A block chain is a linked list that is built with hash pointers instead of pointers.

A use case for a block chain is a **tamper-evident log**. That is, we want to build a log data structure that stores a bunch of data, and allows us to append data onto the end of the log. But if somebody alters data that is earlier in the log, we're going to detect it.

To understand why a block chain achieves this tamper-evident property, let's ask what happens if an adversary wants to tamper with data that's in the middle of the chain. Specifically, the adversary's goal is to do it in such a way that someone who remembers only the hash pointer at the head of the block chain won't be able to detect the tampering. To achieve this goal, the adversary changes the data of some block k . Since the data has been changed, the hash in block $k + 1$, which is a hash of the entire block k , is not going to match up. Remember that we are statistically guaranteed that the new hash will not match the altered content since the hash function is collision resistant. And so we will detect the inconsistency between the new data in block k and the hash pointer in block $k + 1$. Of course the adversary can continue to try and cover up this change by changing the next block's hash as well. The adversary can continue doing this, but this strategy will fail when he reaches the head of the list. Specifically, as long as we store the hash pointer at the head of the list in a place where the adversary cannot change it, the adversary will be unable to change any block without being detected.

The upshot of this is that if the adversary wants to tamper with data anywhere in this entire chain, in order to keep the story consistent, he's going to have to tamper with the hash pointers all the way back to the beginning. And he's ultimately going to run into a roadblock because he won't be able to tamper with the head of the list. Thus it emerges, that by just remembering this single hash pointer, we've essentially remembered a tamper-evident hash of the entire list. So we can build a block chain like this containing as many blocks as we want, going back to some special block at the beginning of the list, which we will call the **genesis block**.

You may have noticed that the block chain construction is similar to the Merkle-Damgard construction that we saw in the previous section. Indeed, they are quite similar, and the same security argument applies to both of them.

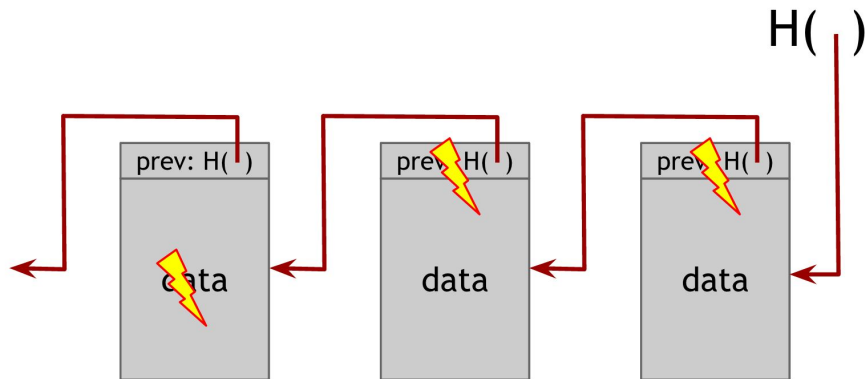


Figure 1.6 Tamper-evident log. If an adversary modifies data anywhere in the block chain, it will result in the hash pointer in the following block being incorrect. If we store the head of the list, then even if the adversary modifies all of the pointers to be consistent with the modified data, the head pointer will be incorrect, and we will detect the tampering.

Merkle trees. Another useful data structure that we can build using hash pointers is a binary tree. A binary tree with hash pointers is known as a **Merkle tree**, after its inventor Ralph Merkle. Suppose we have a number of blocks containing data. These blocks comprise the leaves of our tree. We group these data blocks into pairs of two, and then for each pair, we build a data structure that has two hash pointers, one to each of these blocks. These data structures make the next level up of the tree. We in turn group these into groups of two, and for each pair, create a new data structure that contains the hash of each. We continue doing this until we reach a single block, the root of the tree.

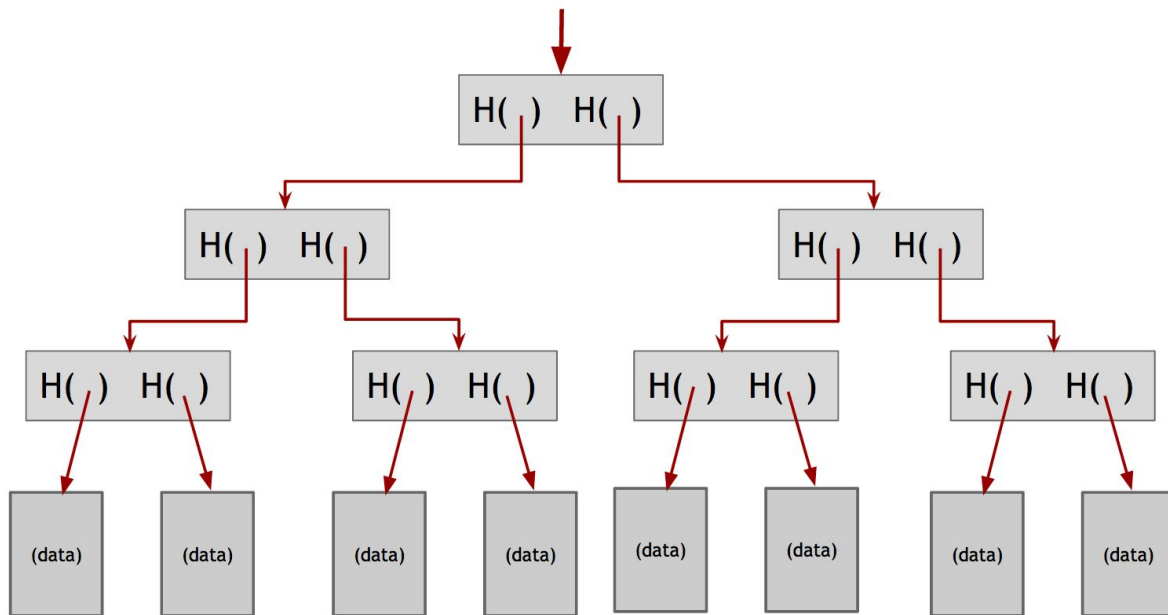


Figure 1.7 Merkle tree. In a Merkle tree, data blocks are grouped in pairs and the hash of each of these blocks is stored in a parent node. The parent nodes are in turn grouped in pairs and their hashes stored one level up the tree. This continues all the way up the tree until we reach the root node.

As before, we remember just the hash pointer at the head of the tree. We now have the ability to traverse down through the hash pointers to any point in the list. This allows us to make sure that the data hasn't been tampered with because, just like we saw with the block chain, if an adversary tampers with some data block at the bottom of the tree, that will cause the hash pointer that's one level up to not match, and even if he continues to tamper with this block, the change will eventually propagate to the top of the tree where he won't be able to tamper with the hash pointer that we've stored. So again, any attempt to tamper with any piece of data will be detected by just remembering the hash pointer at the top.

Proof of membership. Another nice feature of Merkle trees is that, unlike the block chain that we built before, it allows a concise proof of membership. Say that someone wants to prove that a certain data block is a member of the Merkle Tree. As usual, we remember just the root. Then they need to show us this data block, and the blocks on the path from the data block to the root. We can ignore the rest of the tree, as the blocks on this path are enough to allow us to verify the hashes all the way up to the root of the tree. See Figure 1.8 for a graphical depiction of how this works.

If there are n nodes in the tree, only about $\log(n)$ items need to be shown. And since each step just requires computing the hash of the child block, it takes about $\log(n)$ time for us to verify it. And so even if the Merkle tree contains a very large number of blocks, we can still prove membership in a relatively short time. Verification thus runs in time and space that's logarithmic in the number of

nodes in the tree.

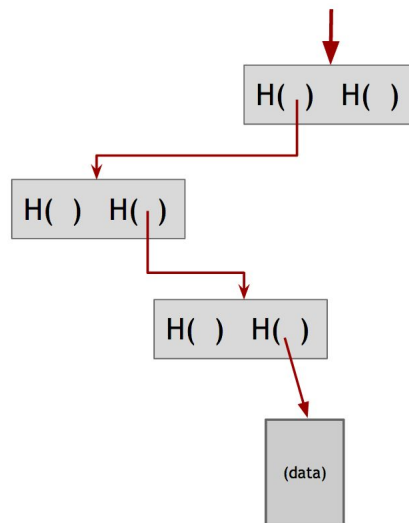


Figure 1.8 Proof of membership. To prove that a data block is included in the tree, one only needs to show the blocks in the path from that data block to the root.

A **sorted Merkle tree** is just a Merkle tree where we take the blocks at the bottom, and we sort them using some ordering function. This can be alphabetical, lexicographical order, numerical order, or some other agreed upon ordering.

Proof of non-membership. With a sorted Merkle tree, it becomes possible to verify non-membership in a logarithmic time and space. That is, we can prove that a particular block is not in the Merkle tree. And the way we do that is simply by showing a path to the item that's just before where the item in question would be and showing the path to the item that is just after where it would be. If these two items are consecutive in the tree, then this serves as a proof that the item in question is not included. For if it was included, it would need to be between the two items shown, but there is no space between them as they are consecutive.

We've discussed using hash pointers in linked lists and binary trees, but more generally, it turns out that we can use hash pointers in any pointer-based data structure as long as the data structure doesn't have cycles. If there are cycles in the data structure, then we won't be able to make all the hashes match up. If you think about it, in an acyclic data structure, we can start near the leaves, or near the things that don't have any pointers coming out of them, compute the hashes of those, and then work our way back toward the beginning. But in a structure with cycles, there's no end we can start with and compute back from.

So, to consider another example, we can build a directed acyclic graph out of hash pointers. And we'll be able to verify membership in that graph very efficiently. And it will be easy to compute. Using hash pointers in this manner is a general trick that you'll see time and again in the context of the distributed data structures and throughout the algorithms that we discuss later in this chapter and

throughout this book.

1.3 Digital Signatures

In this section, we'll look at **digital signatures**. This is the second cryptographic primitive, along with hash functions, that we need as building blocks for the cryptocurrency discussion later on. A digital signature is supposed to be the digital analog to a handwritten signature on paper. We desire two properties from digital signatures that correspond well to the handwritten signature analogy. Firstly, only you can make your signature, but anyone who sees it can verify that it's valid. Secondly, we want the signature to be tied to a particular document so that the signature cannot be used to indicate your agreement or endorsement of a different document. For handwritten signatures, this latter property is analogous to assuring that somebody can't take your signature and snip it off one document and glue it onto the bottom of another one.

How can we build this in a digital form using cryptography? First, let's make the previous intuitive discussion slightly more concrete. This will allow us to reason better about digital signature schemes and discuss their security properties.

Digital signature scheme. A digital signature scheme consists of the following three algorithms:

- **(sk, pk) := generateKeys(keysize)** The generateKeys method takes a key size and generates a key pair. The secret key *sk* is kept privately and used to sign messages. *pk* is the public verification key that you give to everybody. Anyone with this key can verify your signature.
- **sig := sign(sk, message)** The sign method takes a message and a secret key, *sk*, as input and outputs a signature for *message* under *sk*
- **isValid := verify(pk, message, sig)** The verify method takes a message, a signature, and a public key as input. It returns a boolean value, *isValid*, that will be **true** if *sig* is a valid signature for *message* under public key *pk*, and **false** otherwise.

We require that the following two properties hold:

- *Valid signatures must verify*
 $\text{verify}(pk, \text{message}, \text{sign}(sk, \text{message})) == \text{true}$
- Signatures are **existentially unforgeable**

We note that **generateKeys** and **sign** can be randomized algorithms. Indeed, generateKeys had better be randomized because it ought to be generating different keys for different people. **verify**, on the other hand, will always be deterministic.

Let us now examine the two properties that we require of a digital signature scheme in more detail. The first property is straightforward — that valid signatures must verify. If I sign a message with *sk*, my secret key, and someone later tries to validate that signature over that same message using my public key, *pk*, the signature must validate correctly. This property is a basic requirement for signatures to be

useful at all.

Unforgeability. The second requirement is that it's computationally infeasible to forge signatures. That is, an adversary who knows your public key and gets to see your signatures on some other messages can't forge your signature on some message for which he has not seen your signature. This unforgeability property is generally formalized in terms of a game that we play with an adversary. The use of games is quite common in cryptographic security proofs.

In the unforgeability game, there is an adversary who claims that he can forge signatures and a challenger that will test this claim. The first thing we do is we use **generateKeys** to generate a secret signing key and a corresponding public verification key. We give the secret key to the challenger, and we give the public key to both the challenger and to the adversary. So the adversary only knows information that's public, and his mission is to try to forge a message. The challenger knows the secret key. So he can make signatures.

Intuitively, the setup of this game matches real world conditions. A real-life attacker would likely be able to see valid signatures from their would-be victim on a number of different documents. And maybe the attacker could even manipulate the victim into signing innocuous-looking documents if that's useful to the attacker.

To model this in our game, we're going to allow the attacker to get signatures on some documents of his choice, for as long as he wants, as long as the number of guesses is plausible. To give an intuitive idea of what we mean by a plausible number of guesses, we would allow the attacker to try 1 million guesses, but not 2^{80} guesses².

Once the attacker is satisfied that he's seen enough signatures, then the attacker picks some message, M , that they will attempt to forge a signature on. The only restriction on M is that it must be a message for which the attacker has not previously seen a signature (because the attacker can obviously send back a signature that he was given!). The challenger runs the **verify** algorithm to determine if the signature produced by the attacker is a valid signature on M under the public verification key. If it successfully verifies, the attacker wins the game.

² In asymptotic terms, we allow the attacker to try a number of guesses that is a polynomial function of the key size, but no more (e.g. the attacker cannot try exponentially many guesses).

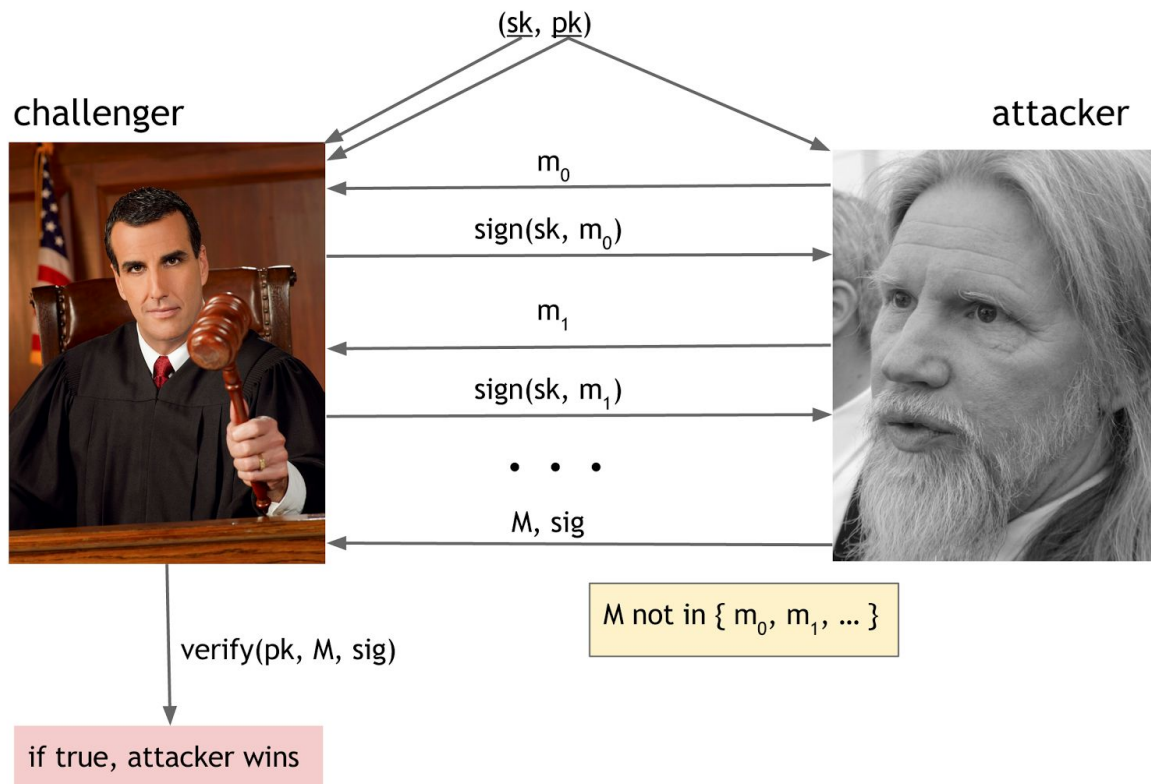


Figure 1.9 Unforgeability game. The adversary and the challenger play the unforgeability game. If the attacker is able to successfully output a signature on a message that he has not previously seen, he wins. If he is unable, the challenger wins and the digital signature scheme is unforgeable.

We say that the signature scheme is unforgeable if and only if, no matter what algorithm the adversary is using, his chance of successfully forging a message is extremely small — so small that we can assume it will never happen in practice.

Practical Concerns. There are a number of practical things that we need to do to turn the algorithmic idea into a digital signature mechanism that can be implemented in practice. For example, many signature algorithms are randomized (in particular the one used in Bitcoin) and we therefore need a good source of randomness. The importance of this really can't be underestimated as bad randomness will make your otherwise-secure algorithm insecure.

Another practical concern is the message size. In practice, there's a limit on the message size that you're able to sign because real schemes are going to operate on bit strings of limited length. There's an easy way around this limitation: sign the hash of the message, rather than the message itself. If we use a cryptographic hash function with a 256-bit output, then we can effectively sign a message of any length as long as our signature scheme can sign 256-bit messages. As we discussed before, it's safe to use the hash of the message as a message digest in this manner since the hash function is collision

resistant.

Another trick that we will use later is that you can sign a hash pointer. If you sign a hash pointer, then the signature covers, or protects, the whole structure — not just the hash pointer itself, but everything the chain of hash pointers points to. For example, if you were to sign the hash pointer that was at the end of a block chain, the result is that you would effectively be digitally signing the entire block chain.

ECDSA. Now let's get into the nuts and bolts. Bitcoin uses a particular digital signature scheme that's called the Elliptic Curve Digital Signature Algorithm (ECDSA). ECDSA is a U.S. government standard, an update of the earlier DSA algorithm adapted to use elliptic curves. These algorithms have received considerable cryptographic analysis over the years and are generally believed to be secure.

More specifically, Bitcoin uses ECDSA over the standard elliptic curve “secp256k1” which is estimated to provide 128 bits of security (that is, it is as difficult to break this algorithm as performing 2^{128} symmetric-key cryptographic operations such as invoking a hash function). While this curve is a published standard, it is rarely used outside of Bitcoin, with other applications using ECDSA (such as key exchange in TLS for secure web browsing) typically using the more common “secp256r1” curve. This is just a quirk of Bitcoin, as this was chosen by Satoshi in the early specification of the system and is now difficult to change.

We won't go into all the details of how ECDSA works as there's some complicated math involved, and understanding it is not necessary for any other content in this book. If you're interested in the details, refer to our further reading section at the end of this chapter. It might be useful to have an idea of the sizes of various quantities, however:

Private key:	256 bits
Public key, uncompressed:	512 bits
Public key, compressed:	257 bits
Message to be signed:	256 bits
Signature:	512 bits

Note that while ECDSA can technically only sign messages 256 bits long, this is not a problem: messages are always hashed before being signed, so effectively any size message can be efficiently signed.

With ECDSA, a good source of randomness is essential because a bad source of randomness will likely leak your key. It makes intuitive sense that if you use bad randomness in generating a key, then the key you generate will likely not be secure. But it's a quirk of ECDSA³ that, even if you use bad

³ For those familiar with DSA, this is a general quirk in DSA and not specific to the elliptic-curve variant.

randomness just in making a signature, using your perfectly good key, that also will leak your private key. And then it's game over; if you leak your private key, an adversary can forge your signature. We thus need to be especially careful about using good randomness in practice, and using a bad source of randomness is a common pitfall of otherwise secure systems.

This completes our discussion of digital signatures as a cryptographic primitive. In the next section, we'll discuss some applications of digital signatures that will turn out to be useful for building cryptocurrencies.

Sidebar: cryptocurrencies and encryption. If you've been waiting to find out which encryption algorithm is used in Bitcoin, we're sorry to disappoint you. There is no encryption in Bitcoin, because nothing needs to be encrypted, as we'll see. Encryption is only one of a rich suite of techniques made possible by modern cryptography. Many of them, such as commitment schemes, involve hiding information in some way, but they are distinct from encryption.

1.4 Public Keys as Identities

Let's look at a nice trick that goes along with digital signatures. The idea is to take a public key, one of those public verification keys from a digital signature scheme, and equate that to an identity of a person or an actor in a system. If you see a message with a signature that verifies correctly under a public key, pk , then you can think of this as pk is saying the message. You can literally think of a public key as kind of like an actor, or a party in a system who can make statements by signing those statements. From this viewpoint, the public key is an identity. In order for someone to speak for the identity pk , they must know the corresponding secret key, sk .

A consequence of treating public keys as identities is that you can make a new identity whenever you want — you simply create a new fresh key pair, sk and pk , via the **generateKeys** operation in our digital signature scheme. pk is the new public identity that you can use, and sk is the corresponding secret key that only you know and lets you speak for on behalf of the identity pk . In practice, you may use the hash of pk as your identity since public keys are large. If you do that, then in order to verify that a message comes from your identity, one will have to check (1) that pk indeed hashes to your identity, and (2) the message verifies under public key pk .

Moreover, by default, your public key pk will basically look random, and nobody will be able to uncover your real world identity by examining pk .⁴ You can generate a fresh identity that looks random, that looks like a face in the crowd, and that only you can control.

Decentralized identity management. This brings us to the idea of decentralized identity management. Rather than having a central authority that you have to go to in order to register as a user in a system, you can register as a user all by yourself. You don't need to be issued a username nor do you need to

⁴ Of course, once you start making statements using this identity, these statements may leak information that allows one to connect pk to your real world identity. We will discuss this in more detail shortly.

inform someone that you're going to be using a particular name. If you want a new identity, you can just generate one at any time, and you can make as many as you want. If you prefer to be known by five different names, no problem! Just make five identities. If you want to be somewhat anonymous for a while, you can make a new identity, use it just for a little while, and then throw it away. All of these things are possible with decentralized identity management, and this is the way Bitcoin, in fact, does identity. These identities are called **addresses**, in Bitcoin jargon. You'll frequently hear the term address used in the context of Bitcoin and cryptocurrencies, and that's really just a hash of a public key. It's an identity that someone made up out of thin air, as part of this decentralized identity management scheme.

Sidebar. The idea that you can generate an identity without a centralized authority may seem counterintuitive. After all, if someone else gets lucky and generates the same key as you can't they steal your bitcoins?

The answer is that the probability of someone else generating the same 256-bit key as you is so small that we don't have to worry about it in practice. We are for all intents and purposes guaranteed that it will never happen.

More generally, in contrast to beginners' intuition that probabilistic systems are unpredictable and hard to reason about, often the opposite is true — the theory of statistics allows us to precisely quantify the chances of events we're interested in and make confident assertions about the behavior of such systems.

But there's a subtlety: the probabilistic guarantee is true only when keys are generated at random. The generation of randomness is often a weak point in real systems. If two users' computers use the same source of randomness or use predictable randomness, then the theoretical guarantees no longer apply. So it is crucial to use a good source of randomness when generating keys to ensure that practical guarantees match the theoretical ones.

On first glance, it may seem that decentralized identity management leads to great anonymity and privacy. After all, you can create a random-looking identity all by yourself without telling anyone your real-world identity. But it's not that simple. Over time, the identity that you create makes a series of statements. People see these statements and thus know that whoever owns this identity has done a certain series of actions. They can start to connect the dots, using this series of actions to infer things about your real-world identity. An observer can link together these things over time, and make inferences that lead them to conclusions such as, "Gee, this person is acting a lot like Joe. Maybe this person is Joe."

In other words, in Bitcoin you don't need to explicitly register or reveal your real-world identity, but the pattern of your behavior might itself be identifying. This is the fundamental privacy question in a cryptocurrency like Bitcoin, and indeed we'll devote the entirety of Chapter 6 to it.

1.5 A Simple Cryptocurrency

Now let's move from cryptography to cryptocurrencies. Eating our cryptographic vegetables will start to pay off here, and we'll gradually see how the pieces fit together and why cryptographic operations like hash functions and digital signatures are actually useful. In this section we'll discuss two very simple cryptocurrencies. Of course, it's going to require much of the rest of the book to spell out all the implications of how Bitcoin itself works.

GoofyCoin

The first of the two is GoofyCoin, which is about the simplest cryptocurrency we can imagine. There are just two rules of GoofyCoin. The first rule is that a designated entity, Goofy, can create new coins whenever he wants and these newly created coins belong to him.

To create a coin, Goofy generates a unique coin ID `uniqueCoinID` that he's never generated before and constructs the string `"CreateCoin [uniqueCoinID]"`. He then computes the digital signature of this string with his secret signing key. The string, together with Goofy's signature, is a coin. Anyone can verify that the coin contains Goofy's valid signature of a `CreateCoin` statement, and is therefore a valid coin.

The second rule of GoofyCoin is that whoever owns a coin can transfer it on to someone else. Transferring a coin is not simply a matter of sending the coin data structure to the recipient — it's done using cryptographic operations.

Let's say Goofy wants to transfer a coin that he created to Alice. To do this he creates a new statement that says `"Pay this to Alice"` where `"this"` is a hash pointer that references the coin in question. And as we saw earlier, identities are really just public keys, so `"Alice"` refers to Alice's public key. Finally, Goofy signs the string representing the statement. Since Goofy is the one who originally owned that coin, he has to sign any transaction that spends the coin. Once this data structure representing Goofy's transaction signed by him exists, Alice owns the coin. She can prove to anyone that she owns the coin, because she can present the data structure with Goofy's valid signature. Furthermore, it points to a valid coin that was owned by Goofy. So the validity and ownership of coins are self-evident in the system.

Once Alice owns the coin, she can spend it in turn. To do this she creates a statement that says, `"Pay this coin to Bob's public key"` where `"this"` is a hash pointer to the coin that was owned by her. And of course, Alice signs this statement. Anyone, when presented with this coin, can verify that Bob is the owner. They would follow the chain of hash pointers back to the coin's creation and verify that at each step, the rightful owner signed a statement that says `"pay this coin to [new owner]"`.

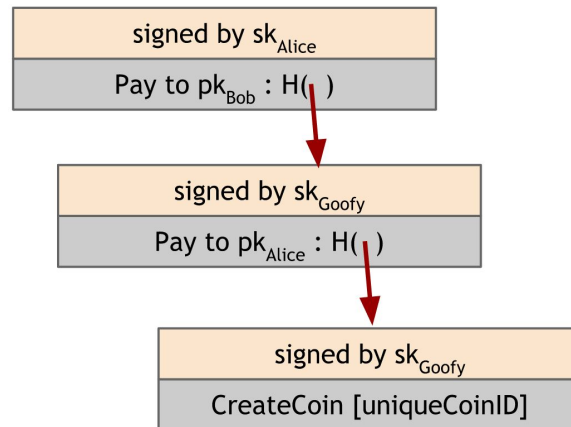


Figure 1.10 GoofyCoin coin. Shown here is a coin that's been created (bottom) and spent twice (middle and top).

To summarize, the rules of GoofyCoin are:

- Goofy can create new coins by simply signing a statement that he's making a new coin with a unique coin ID.
- Whoever owns a coin can pass it on to someone else by signing a statement that saying, "Pass on this coin to X" (where X is specified as a public key)
- Anyone can verify the validity of a coin by following the chain of hash pointers back to its creation by Goofy, verifying all of the signatures along the way.

Of course, there's a fundamental security problem with GoofyCoin. Let's say Alice passed her coin on to Bob by sending her signed statement to Bob but didn't tell anyone else. She could create another signed statement that pays the very same coin to Chuck. To Chuck, it would appear that it is perfectly valid transaction, and now he's the owner of the coin. Bob and Chuck would both have valid-looking claims to be the owner of this coin. This is called a double-spending attack — Alice is spending the same coin twice. Intuitively, we know coins are not supposed to work that way.

In fact, double-spending attacks are one of the key problems that any cryptocurrency has to solve. GoofyCoin does not solve the double-spending attack and therefore it's not secure. GoofyCoin is simple, and its mechanism for transferring coins is actually very similar to Bitcoin, but because it is insecure it won't cut it as a cryptocurrency.

ScroogeCoin

To solve the double-spending problem, we'll design another cryptocurrency, which we'll call ScroogeCoin. ScroogeCoin is built off of GoofyCoin, but it's a bit more complicated in terms of data structures.

The first key idea is that a designated entity called Scrooge publishes an *append-only ledger* containing the history of all the transactions that have happened. The append-only property ensures that any data written to this ledger will remain forever. If the ledger is truly append-only, we can use it to defend against double-spending by requiring all transactions to be written the ledger before they are accepted. That way, it will be publicly visible if coins were previously sent to a different owner.

To implement this append-only functionality, Scrooge can build a block chain (the data structure we discussed before) which he will digitally sign. It's a series of data blocks, each with one transaction in it (in practice, as an optimization, we'd really put multiple transactions into the same block, as Bitcoin does.) Each block has the ID of a transaction, the transaction's contents, and a hash pointer to the previous block. Scrooge digitally signs the final hash pointer, which binds all of the data in this entire structure, and publishes the signature along with the block chain.

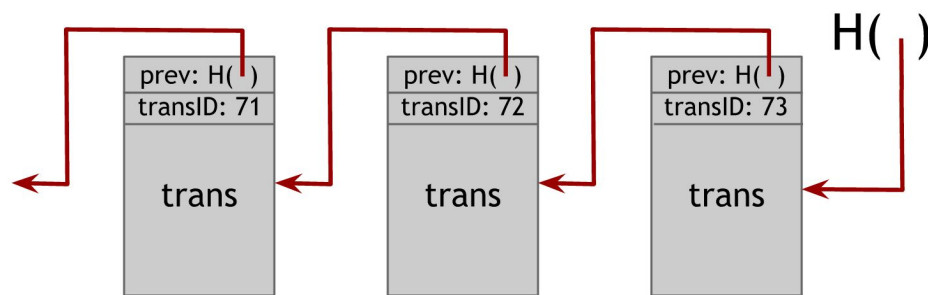


Figure 1.11 ScroogeCoin block chain.

In ScroogeCoin a transaction only counts if it is in the block chain signed by Scrooge. Anybody can verify that a transaction was endorsed by Scrooge by checking Scrooge's signature on the block that it appears in. Scrooge makes sure that he doesn't endorse a transaction that attempts to double-spend an already spent coin.

Why do we need a block chain with hash pointers in addition to having Scrooge sign each block? This ensures the append-only property. If Scrooge tries to add or remove a transaction to the history, or change an existing transaction, it will affect all of the following blocks because of the hash pointers. As long as someone is monitoring the latest hash pointer published by Scrooge, the change will be obvious and easy to catch. In a system where Scrooge signed blocks individually, you'd have to keep track of every single signature Scrooge ever issued. A block chain makes it very easy for any two individuals to verify that they have observed the exact same history of transactions signed by Scrooge.

In ScroogeCoin, there are two kinds of transactions. The first kind is CreateCoins, which is just like the operation Goofy could do in GoofyCoin that makes a new coin. With ScroogeCoin, we'll extend the semantics a bit to allow multiple coins to be created in one transaction.

transID: 73 type:CreateCoins		
coins created		
<i>num</i>	<i>value</i>	<i>recipient</i>
0	3.2	0x...
1	1.4	0x...
2	7.1	0x...

← coinID 73(0)

← coinID 73(1)

← coinID 73(2)

Figure 1.12 CreateCoins transaction. This CreateCoins transaction creates multiple coins. Each coin has a serial number within the transaction. Each coin also has a value; it's worth a certain number of scrooge coins. Finally, each coin has a recipient, which is a public key that gets the coin when it's created. So CreateCoins creates a bunch of new coins with different values and assigns them to people as initial owners. We refer to coins by CoinIDs. A CoinID is a combination of a transaction ID and the coin's serial number within that transaction.

A CreateCoins transaction is always valid by definition if it is signed by Scrooge. We won't worry about when Scrooge is entitled to create coins or how many, just like we didn't worry in GoofyCoin about how Goofy is chosen as the entity allowed to create coins.

The second kind of transaction is PayCoins. It consumes some coins, that is, destroys them, and creates new coins of the same total value. The new coins might belong to different people (public keys). This transaction has to be signed by everyone who's paying in a coin. So if you're the owner of one of the coins that's going to be consumed in this transaction, then you need to digitally sign the transaction to say that you're really okay with spending this coin.

The rules of ScroogeCoin say that PayCoins transaction is valid if four things are true:

- The consumed coins are valid, that is, they really were created in previous transactions.
- The consumed coins were not already consumed in some previous transaction. That is, that this is not a double-spend.
- The total value of the coins that come out of this transaction is equal to the total value of the coins that went in. That is, only Scrooge can create new value.
- The transaction is validly signed by the owners of all of the consumed coins.

transID: 73 type:PayCoins		
consumed coinIDs: 68(1), 42(0), 72(3)		
coins created		
<i>num</i>	<i>value</i>	<i>recipient</i>
0	3.2	0x...
1	1.4	0x...
2	7.1	0x...
signatures		

Figure 1.13 A PayCoins Transaction.

If all of those conditions are met, then this PayCoins transaction is valid and Scrooge will accept it. He'll write it into the history by appending it to the block chain, after which everyone can see that this transaction has happened. It is only at this point that the participants can accept that the transaction has actually occurred. Until it is published, it might be preempted by a double-spending transaction even if it is otherwise valid by the first three conditions.

Coins in this system are immutable — they are never changed, subdivided, or combined. Each coin is created, once, in one transaction and later consumed in some other transaction. But we can get the same effect as being able to subdivide or combine coins by using transactions. For example, to subdivide a coin, Alice create a new transaction that consumes that one coin, and then produces two new coins of the same total value. Those two new coins could be assigned back to her. So although coins are immutable in this system, it has all the flexibility of a system that didn't have immutable coins.

Now, we come to the core problem with ScroogeCoin. ScroogeCoin will work in the sense that people can see which coins are valid. It prevents double-spending, because everyone can look into the block chain and see that all of the transactions are valid and that every coin is consumed only once. But the problem is Scrooge — he has too much influence. He can't create fake transactions, because he can't forge other people's signatures. But he could stop endorsing transactions from some users, denying them service and making their coins unspendable. If Scrooge is greedy (as his cartoon namesake suggests) he could refuse to publish transactions unless they transfer some mandated transaction fee to him. Scrooge can also of course create as many new coins for himself as he wants. Or Scrooge could get bored of the whole system and stop updating the block chain completely.

The problem here is centralization. Although Scrooge is happy with this system, we, as users of it, might not be. While ScroogeCoin may seem like an unrealistic proposal, much of the early research on cryptosystems assumed there would indeed be some central trusted authority, typically referred to as a *bank*. After all, most real-world currencies do have a trusted issuer (typically a government mint) responsible for creating currency and determining which notes are valid. However, cryptocurrencies with a central authority largely failed to take off in practice. There are many reasons for this, but in hindsight it appears that it's difficult to get people to accept a cryptocurrency with a centralized authority.

Therefore, the central technical challenge that we need to solve in order to improve on ScroogeCoin and create a workable system is: can we descroogify the system? That is, can we get rid of that centralized Scrooge figure? Can we have a cryptocurrency that operates like ScroogeCoin in many ways, but doesn't have any central trusted authority?

To do that, we need to figure out how all users can agree upon a single published block chain as the history of which transactions have happened. They must all agree on which transactions are valid, and which transactions have actually occurred. They also need to be able to assign IDs to things in a decentralized way. Finally, the minting of new coins needs to be controlled in a decentralized way. If we can solve all of those problems, then we can build a currency that would be like ScroogeCoin but without a centralized party. In fact, this would be a system very much like Bitcoin.

Further Reading

Steven Levy's *Crypto* is an enjoyable, non-technical look at the development of modern cryptography and the people behind it:

Levy, Steven. *Crypto: How the Code Rebels Beat the Government--Saving Privacy in the Digital Age*. Penguin, 2001.

Modern cryptography is a rather theoretical field. Cryptographers use mathematics to define primitives, protocols, and their desired security properties in a formal way, and to prove them secure based on widely accepted assumptions about the computational hardness of specific mathematical tasks. In this chapter we've used intuitive language to discuss hash functions and digital signatures. For the reader interested in exploring these and other cryptographic concepts in a more mathematical way and in greater detail, we refer you to:

Katz, Jonathan, and Yehuda Lindell. *Introduction to Modern Cryptography, Second Edition*. CRC Press, 2014.

For an introduction to applied cryptography, see:

Ferguson, Niels, Bruce Schneier, and Tadayoshi Kohno. *Cryptography engineering: design principles and practical applications*. John Wiley & Sons, 2012.

Perusing the NIST standard that defines SHA-2 is a good way to get an intuition for what cryptographic standards look like:

[FIPS PUB 180-4, Secure Hash Standards](#), Federal Information Processing Standards Publication. Information Technology Laboratory, National Institute of Standards and Technology, Gaithersburg, MD, 2008.

Finally, here's the paper describing the standardized version of the ECDSA signature algorithm.

Johnson, Don, Alfred Menezes, and Scott Vanstone. [The elliptic curve digital signature algorithm \(ECDSA\)](#). International Journal of Information Security 1.1 (2001): 36-63.

Exercises

1. **Authenticated Data Structures.** You are designing SecureBox, an authenticated online file storage system. For simplicity, there is only a single folder. Users must be able to add, edit, delete, and retrieve files, and to list the folder contents. When a user retrieves a file, SecureBox must provide a proof that the file hasn't been tampered with since its last update. If a file with the given name doesn't exist, the server must report that — again with a proof.

We want to minimize the size of these proofs, the time complexity of verifying them, and the size of the digest that the user must store between operations. (Naturally, to be able to verify proofs, users must at all times store some nonzero amount of state derived from the folder contents. Other than this digest the user has no memory of the contents of the files she added.)

Here's a naive approach. The user's digest is a hash of the entire folder contents, and proofs are copies of the entire folder contents. This results in a small digest but large proofs and long verification times. Besides, before executing add/delete/edit operations, the user must retrieve the entire folder so that she can recompute the digest.

Alternatively, the digest could consist of a separate hash for each file, and each file would be its own proof. The downside of this approach is that it requires digest space that is linear in the number of files in the system.

Can you devise a protocol where proof size, verification time, and digest size are all sublinear? You might need a sub-protocol that involves some amount of two-way communication for the user to be able to update her digest when she executes and add, delete, or edit.

Hint: use the Merkle tree idea from Section 1.2.

2. **Birthday Attack.** Let H be an ideal hash function that produces an n -bit output. By ideal, we mean that as far as we can tell, each hash value is independent and uniformly distributed in $\{0,1\}^n$. Trivially,

we can go through $2^n + 1$ different values and we are guaranteed to find a collision. If we're constrained for space, we can just store 1 input-output pair and keep trying new inputs until we hit the same output again. This has time complexity $O(2^n)$, but has $O(1)$ space complexity. Alternatively, we could compute the hashes of about $O(2^{n/2})$ different inputs and store all the input-output pairs. As we saw in the text, there's a good chance that some two of those outputs would collide (the "birthday paradox"). This shows that we can achieve a time-space trade-off: $O(2^{n/2})$ time and $O(2^{n/2})$ space.

1. (Easy) Show that the time-space trade-off is parameterizable: we can achieve any space complexity between $O(1)$ and $O(2^{n/2})$ with a corresponding decrease in time complexity.
2. (Very hard) Is there an attack for which the product of time and space complexity is $o(2^n)$?
[Recall the little oh notation.]

3. **Hash function properties** (again). Let H be a hash function that is both hiding and puzzle-friendly. Consider $G(z) = H(z) \parallel z_{\text{last}}$ where z_{last} represents the last bit of z . Show that G is puzzle-friendly but not hiding.

4. **Randomness**. In ScroogeCoin, suppose Mallory tries generating (sk, pk) pairs until her secret key matches someone else's. What will she be able to do? How long will it take before she succeeds, on average? What if Alice's random number generator has a bug and her key generation procedure produces only 1,000 distinct pairs?

Chapter 2: How Bitcoin Achieves Decentralization

In this chapter, we will discuss decentralization in Bitcoin. In the first chapter we looked at the crypto basics that underlie Bitcoin and we ended with a simple currency that we called ScroogeCoin. ScroogeCoin achieves a lot of what we want in a ledger-based cryptocurrency, but it has one glaring problem — it relies upon the centralized authority called Scrooge. We ended with the question of how to decentralize, or de-Scrooge-ify, this currency, and answering that question will be the focus of this chapter.

As you read through this chapter, take note that the mechanism through which Bitcoin achieves decentralization is not purely technical, but it's a combination of technical methods and clever incentive engineering. At the end of this chapter you should have a really good appreciation for how this decentralization happens, and more generally how Bitcoin works and why it is secure.

2.1 Centralization vs. Decentralization

Decentralization is an important concept that is not unique to Bitcoin. The notion of competing paradigms of centralization versus decentralization arises in a variety of different digital technologies. In order to best understand how it plays out in Bitcoin, it is useful to understand the central conflict — the tension between these two paradigms — in a variety of other contexts.

On the one hand we have the Internet, a famously decentralized system that has historically competed with and prevailed against “walled-garden” alternatives like AOL's and CompuServe's information services. Then there's email, which at its core is a decentralized system based on the Simple Mail Transfer Protocol (SMTP), an open standard. While it does have competition from proprietary messaging systems like Facebook or LinkedIn mail, email has managed to remain the default for person-to-person communication online. In the case of instant messaging and text messaging, we have a hybrid model that can't be categorically described as centralized or decentralized. Finally there's social networking: despite numerous concerted efforts by hobbyists, developers and entrepreneurs to create alternatives to the dominant centralized model, centralized systems like Facebook and LinkedIn still dominate this space. In fact, this conflict long predates the digital era and we see a similar struggle between the two models in the history of telephony, radio, television, and film.

Decentralization is not all or nothing; almost no system is purely decentralized or purely centralized. For example, email is fundamentally a decentralized system based on a standardized protocol, SMTP, and anyone who wishes can operate an email server of their own. Yet, what has happened in the market is that a small number of centralized webmail providers have become dominant. Similarly, while the Bitcoin protocol is decentralized, services like Bitcoin exchanges, where you can convert

Bitcoin and Cryptocurrency Technologies

**Arvind Narayanan, Joseph Bonneau, Edward Felten,
Andrew Miller, Steven Goldfeder**

with a preface by Jeremy Clark

Draft — Feb 9, 2016

Feedback welcome! Email bitcoinbook@lists.cs.princeton.edu

For the latest draft and supplementary materials including programming assignments,
see our [Coursera course](#).

The official version of this book will be published by Princeton University Press in 2016.
If you'd like to be notified when it's available, please sign up [here](#).

Chapter 2: How Bitcoin Achieves Decentralization

In this chapter, we will discuss decentralization in Bitcoin. In the first chapter we looked at the crypto basics that underlie Bitcoin and we ended with a simple currency that we called ScroogeCoin. ScroogeCoin achieves a lot of what we want in a ledger-based cryptocurrency, but it has one glaring problem — it relies upon the centralized authority called Scrooge. We ended with the question of how to decentralize, or de-Scrooge-ify, this currency, and answering that question will be the focus of this chapter.

As you read through this chapter, take note that the mechanism through which Bitcoin achieves decentralization is not purely technical, but it's a combination of technical methods and clever incentive engineering. At the end of this chapter you should have a really good appreciation for how this decentralization happens, and more generally how Bitcoin works and why it is secure.

2.1 Centralization vs. Decentralization

Decentralization is an important concept that is not unique to Bitcoin. The notion of competing paradigms of centralization versus decentralization arises in a variety of different digital technologies. In order to best understand how it plays out in Bitcoin, it is useful to understand the central conflict — the tension between these two paradigms — in a variety of other contexts.

On the one hand we have the Internet, a famously decentralized system that has historically competed with and prevailed against “walled-garden” alternatives like AOL's and CompuServe's information services. Then there's email, which at its core is a decentralized system based on the Simple Mail Transfer Protocol (SMTP), an open standard. While it does have competition from proprietary messaging systems like Facebook or LinkedIn mail, email has managed to remain the default for person-to-person communication online. In the case of instant messaging and text messaging, we have a hybrid model that can't be categorically described as centralized or decentralized. Finally there's social networking: despite numerous concerted efforts by hobbyists, developers and entrepreneurs to create alternatives to the dominant centralized model, centralized systems like Facebook and LinkedIn still dominate this space. In fact, this conflict long predates the digital era and we see a similar struggle between the two models in the history of telephony, radio, television, and film.

Decentralization is not all or nothing; almost no system is purely decentralized or purely centralized. For example, email is fundamentally a decentralized system based on a standardized protocol, SMTP, and anyone who wishes can operate an email server of their own. Yet, what has happened in the market is that a small number of centralized webmail providers have become dominant. Similarly, while the Bitcoin protocol is decentralized, services like Bitcoin exchanges, where you can convert

Bitcoin into other currencies, and wallet software, or software that allows people to manage their bitcoins may be centralized or decentralized to varying degrees.

With this in mind, let's break down the question of how the Bitcoin protocol achieves decentralization into five more specific questions:

1. Who maintains the ledger of transactions?
2. Who has authority over which transactions are valid?
3. Who creates new bitcoins?
4. Who determines how the rules of the system change?
5. How do bitcoins acquire exchange value?

The first three questions reflect the technical details of the Bitcoin protocol, and it is these questions that will be the focus of this chapter.

Different aspects of Bitcoin fall on different points on the centralization/decentralization spectrum. The peer-to-peer network is close to purely decentralized since anybody can run a Bitcoin node and there's a fairly low barrier to entry. You can go online and easily download a Bitcoin client and run a node on your laptop or your PC. Currently there are several thousand such nodes. Bitcoin *mining*, which we'll study later in this chapter, is technically also open to anyone, but it requires a very high capital cost. Because of this there has been a high degree of centralization, or a concentration of power, in the Bitcoin mining ecosystem. Many in the Bitcoin community see this as quite undesirable. A third aspect is updates to the software that Bitcoin nodes run, and this has a bearing on how and when the rules of the system change. One can imagine that there are numerous interoperable implementations of the protocol, as with email. But in practice, most nodes run the reference implementation, and its developers are trusted by the community and have a lot of power.

2.2 Distributed consensus

We've discussed, in a generic manner, centralization and decentralization. Let's now examine decentralization in Bitcoin at a more technical level. A key term that will come up throughout this discussion is **consensus**, and specifically, **distributed consensus**. The key technical problem to solve in building a distributed e-cash system is achieving distributed consensus. Intuitively, you can think of our goal as decentralizing ScroogeCoin, the hypothetical currency that we saw in the first chapter.

Distributed consensus has various applications, and it has been studied for decades in computer science. The traditional motivating application is reliability in distributed systems. Imagine you're in charge of the backend for a large social networking company like Facebook. Systems of this sort typically have thousands or even millions of servers, which together form a massive distributed database that records all of the actions that happen in the system. Each piece of information must be recorded on several different nodes in this backend, and the nodes must be in sync about the overall

state of the system.

The implications of having a distributed consensus protocol reach far beyond this traditional application. If we had such a protocol, we could use it to build a massive, distributed key-value store, that maps arbitrary keys, or names, to arbitrary values. A distributed key-value store, in turn, would enable many applications. For example, we could use it to build a distributive domain name system, which is simply a mapping between human understandable domain names to IP addresses. We could build a public key directory, which is a mapping between email addresses (or some other form of real-world identity) to public keys.

That's the intuition of what distributed consensus is, but it is useful to provide a technical definition as this will help us determine whether or not a given protocol meets the requirements.

Distributed consensus protocol. There are n nodes that each have an input value. Some of these nodes are faulty or malicious. A distributed consensus protocol has the following two properties:

- It must terminate with all honest nodes in agreement on the value
- The value must have been generated by an honest node

What does this mean in the context of Bitcoin? To understand how distributed consensus could work in Bitcoin, remember that Bitcoin is a peer-to-peer system. When Alice wants to pay Bob, what she actually does is broadcast a transaction to all of the Bitcoin nodes that comprise the peer-to-peer network. See Figure 2.1.

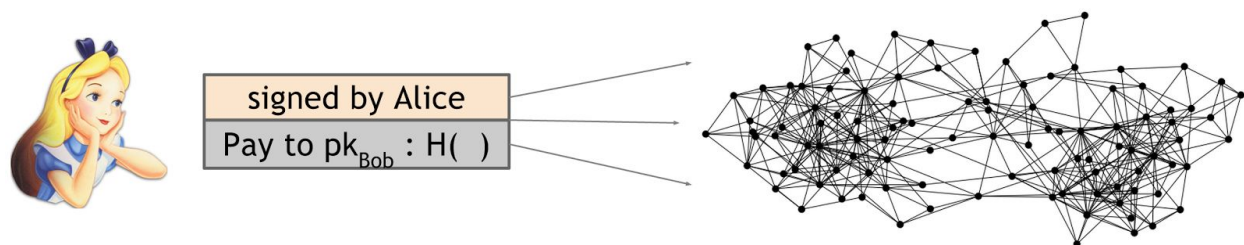


Figure 2.1 Broadcasting a transaction In order to pay Bob, Alice broadcasts the transaction to the entire Bitcoin peer-to-peer network.

Incidentally, you may have noticed that Alice broadcasts the transaction to all the Bitcoin peer-to-peer nodes, but Bob's computer is nowhere in this picture. It's of course possible that Bob is running one of the nodes in the peer-to-peer network. In fact, if he wants to be notified that this transaction did in fact happen and that he got paid, running a node might be a good idea. Nevertheless, there is no requirement that Bob be listening on the network; running a node is not necessary for Bob to receive the funds. The bitcoins will be his whether or not he's operating a node on the network.

What exactly is it that the nodes might want to reach consensus on in the Bitcoin network? Given that a variety of users are broadcasting these transactions to the network, the nodes must agree on

exactly which transactions were broadcast and the order in which these transactions happened. This will result in a single, global ledger for the system. Recall that in ScroogeCoin, for optimization, we put transactions into blocks. Similarly, in Bitcoin, we do consensus on a block-by-block basis.

So at any given point, all the nodes in the peer-to-peer network have a ledger consisting of a sequence of blocks, each containing a list of transactions, that they've reached consensus on. Additionally, each node has a pool of outstanding transactions that it has heard about but have not yet been included on the block chain. For these transactions, consensus has not yet happened, and so by definition, each node might have a slightly different version of the outstanding transaction pool. In practice, this occurs because the peer-to-peer network is not perfect, so some nodes may have heard about a transaction that other nodes have not heard about.

How exactly do nodes come to consensus on a block? One way to do this: at regular intervals, say every 10 minutes, every node in the system proposes its own outstanding transaction pool to be the next block. Then the nodes execute some consensus protocol, where each node's input is its own proposed block. Now, some nodes may be malicious and put invalid transactions into their blocks, but we might assume that other nodes will be honest. If the consensus protocol succeeds, a valid block will be selected as the output. Even if the selected block was proposed by only one node, it's a valid output as long as the block is valid. Now there may be some valid outstanding transaction that did not get included in the block, but this is not a problem. If some transaction somehow didn't make it into this particular block, it could just wait and get into the next block.

The approach in the previous paragraph has some similarities to how Bitcoin works, but it's not quite how it works. There are a number of technical problems with this approach. Firstly, consensus in general is a hard problem since nodes might crash or be outright malicious. Secondly, and specifically in the Bitcoin context, the network is highly imperfect. It's a peer-to-peer system, and not all pairs of nodes are connected to each other. There could be faults in the network because of poor Internet connectivity for example, and thus running a consensus protocol in which all nodes must participate is not really possible. Finally, there's a lot of latency in the system because it's distributed all over the Internet.

Sidebar: The Bitcoin protocol must reach consensus in the face of two types of obstacles: imperfections in the network, such as latency and nodes crashing, as well as deliberate attempts by some nodes to subvert the process.

One particular consequence of this high latency is that there is no notion of global time. What this means is that not all nodes can agree to a common ordering of events simply based on observing timestamps. So the consensus protocol cannot contain instructions of the form, "The node that sent the first message in step 1 must do X in step 2." This simply will not work because not all nodes will agree on which message was sent first in the step 1 of the protocol.

Impossibility results. The lack of global time heavily constrains the set of algorithms that can be used in the consensus protocols. In fact, because of these constraints, much of the literature on distributed

consensus is somewhat pessimistic, and many impossibility results have been proven. One very well known impossibility result concerns the **Byzantine Generals Problem**. In this classic problem, the Byzantine army is separated into divisions, each commanded by a general. The generals communicate by messenger in order to devise a joint plan of action. Some generals may be traitors and may intentionally try to subvert the process so that the loyal generals cannot arrive at a unified plan. The goal of this problem is for all of the loyal generals to arrive at the same plan without the traitorous generals being able to cause them to adopt a bad plan. It has been proven that this is impossible to achieve if one-third or more of the generals are traitors.

A much more subtle impossibility result, known for the names of the authors who first proved it, is called the Fischer-Lynch-Paterson impossibility result. Under some conditions, which include the nodes acting in a deterministic manner, they proved that consensus is impossible with even a single faulty process.

Despite these impossibility results, there are some consensus protocols in the literature. One of the better known among these protocols is **Paxos**. Paxos makes certain compromises. On the one hand, it never produces an inconsistent result. On the other hand, it accepts the trade-off that under certain conditions, albeit rare ones, the protocol can get stuck and fail to make any progress.

Breaking traditional assumptions. But there's good news: these impossibility results were proven in a very specific model. They were intended to study distributed databases, and this model doesn't carry over very well to the Bitcoin setting because Bitcoin violates many of the assumptions built into the models. In a way, the results tell us more about the model than they do about the problem of distributed consensus.

Ironically, with the current state of research, consensus in Bitcoin works better in practice than in theory. That is, we observe consensus working, but have not developed the theory to fully explain why it works. But developing such a theory is important as it can help us predict unforeseen attacks and problems, and only when we have a strong theoretical understanding of how Bitcoin consensus works will we have strong guarantees Bitcoin's security and stability.

What are the assumptions in traditional models for consensus that Bitcoin violates? First, it introduces the idea of incentives, which is novel for a distributed consensus protocol. This is only possible in Bitcoin because it is a currency and therefore has a natural mechanism to incentivize participants to act honestly. So Bitcoin doesn't quite solve the distributed consensus problem in a general sense, but it solves it in the specific context of a currency system.

Second, Bitcoin embraces the notion of randomness. As we will see in the next two sections, Bitcoin's consensus algorithm relies heavily on randomization. Also, it does away with the notion of a specific starting point and ending point for consensus. Instead, consensus happens over a long period of time, about an hour in the practical system. But even at the end of that time, nodes can't be certain that any particular transaction or a block has made it into the ledger. Instead, as time goes on, the probability that your view of any block will match the eventual consensus view increases, and the

probability that the views will diverge goes down exponentially. These differences in the model are key to how Bitcoin gets around the traditional impossibility results for distributed consensus protocols.

2.3 Consensus without identity using a block chain

In this section we'll study the technical details of Bitcoin's consensus algorithm. Recall that Bitcoin nodes do not have persistent, long-term identities. This is another difference from traditional distributed consensus algorithms. One reason for this lack of identities is that in a peer-to-peer system, there is no central authority to assign identities to participants and verify that they're not creating new nodes at will. The technical term for this is a **Sybil attack**. Sybils are just copies of nodes that a malicious adversary can create to look like there are a lot of different participants, when in fact all those pseudo-participants are really controlled by the same adversary. The other reason is that pseudonymity is inherently a goal of Bitcoin. Even if it were possible or easy to establish identities for all nodes or all participants, we wouldn't necessarily want to do that. Although Bitcoin doesn't give strong anonymity guarantees in that the different transactions that one makes can often be linked together, it does have the property that nobody is forced to reveal their real-life identity, like their name or IP address, in order to participate. And that's an important property and a central feature of Bitcoin's design.

If nodes did have identities, the design would be easier. For starters, identities would allow us to put in the protocol instructions of the form, "Now the node with the lowest numerical ID should take some step." Without identities, the set of possible instructions is more constrained. But a much more serious reason for nodes to have identities is for security. If nodes were identified and it weren't trivial to create new node identities, then we could make assumptions on the number of nodes that are malicious, and we could derive security properties out of that. For both of these reasons, the lack of identities introduces difficulties for the consensus protocol in Bitcoin.

We can compensate for the lack of identities by making a weaker assumption. Suppose there is somehow an ability to pick a random node in the system. A good motivating analogy for this is a lottery or a raffle, or any number of real-life systems where it's hard to track people, give them identities and verify those identities. What we do in those contexts is to give out tokens or tickets or something similar. That enables us to later pick a random token ID, and call upon the owner of that ID. So for the moment, take a leap of faith and assume that it is possible to pick a random node from the Bitcoin network in this manner. Further assume, for the moment, that this token generation and distribution algorithm is sufficiently smart so that if the adversary is going to try to create a lot of Sybil nodes, all of those Sybils together will get only one token. This means the adversary is not able to multiply his power by creating new nodes. If you think this is a lot to assume, don't worry. Later in this chapter, we'll remove these assumptions and show in detail how properties equivalent to these are realized in Bitcoin.

Implicit Consensus. This assumption of random node selection makes possible something called **implicit consensus**. There are multiple rounds in our protocol, each corresponding to a different block in the block chain. In each round, a random node is somehow selected, and this node gets to propose the next block in the chain. There is no consensus algorithm for selecting the block, and no voting of any kind. The chosen node unilaterally proposes what the next block in the block chain will be. But what if that node is malicious? Well, there is a process for handling that, but it is an implicit one. Other nodes will implicitly accept or reject that block by choosing whether or not to build on top of it. If they accept that block, they will signal their acceptance by extending the block chain including the accepted block. By contrast, if they reject that block, they will extend the chain by ignoring that block, and building on top of whichever is the previous block that they accepted. Recall that each block contains a hash of the block that it extends. This is the technical mechanism that allows nodes to signal which block it is that they are extending.

Bitcoin consensus algorithm (simplified)

This algorithm is simplified in that it assumes the ability to select a random node in a manner that is not vulnerable to Sybil attacks.

1. New transactions are broadcast to all nodes
2. Each node collects new transactions into a block
3. In each round a random node gets to broadcast its block
4. Other nodes accept the block only if all transactions in it are valid (unspent, valid signatures)
5. Nodes express their acceptance of the block by including its hash in the next block they create

Let's now try to understand why this consensus algorithm works. To do this, let's consider how a malicious adversary — who we'll call Alice — may be able to subvert this process.

Stealing Bitcoins. Can Alice simply steal bitcoins belonging to another user at an address she doesn't control? No. Even if it is Alice's turn to propose the next block in the chain, she cannot steal other users' bitcoins. Doing so would require Alice to create a valid transaction that spends that coin. This would require Alice to forge the owners' signatures which she cannot do if a secure digital signature scheme is used. So as long as the underlying cryptography is solid, she's not able to simply steal bitcoins.

Denial of service attack. Let's consider another attack. Say Alice really dislikes some other user Bob. Alice can then decide that she will not include any transactions originating from Bob's address in any block that she proposes to get onto the block chain. In other words, she's denying service to Bob. While this is a valid attack that Alice can try to mount, luckily it's nothing more than a minor

annoyance. If Bob's transaction doesn't make it into the next block that Alice proposes, he will just wait until an honest node gets the chance to propose a block and then his transaction will get into that block. So that's not really a good attack either.

Double-spend attack. Alice may try to launch a double-spend attack. To understand how that works, let's assume that Alice is a customer of some online merchant or website run by Bob, who provides some online service in exchange for payment in bitcoins. Let's say Bob's service allows the download of some software. So here's how a double-spend attack might work. Alice adds an item to her shopping cart on Bob's website and the server requests payment. Then Alice creates a Bitcoin transaction from her address to Bob's and broadcasts it to the network. Let's say that some honest node creates the next block, and includes this transaction in that block. So there is now a block that was created by an honest node that contains a transaction that represents a payment from Alice to the merchant Bob.

Recall that a transaction is a data structure that contains Alice's signature, an instruction to pay to Bob's public key, and a hash. This hash represents a pointer to a previous transaction output that Alice received and is now spending. That pointer must reference a transaction that was included in some previous block in the consensus chain.

Note, by the way, that there are two different types of hash pointers here that can easily be confused. Blocks include a hash pointer to the previous block that they're extending. Transactions include one or more hash pointers to previous transaction outputs that are being redeemed.

Let's return to how Alice can launch a double spend attack. The latest block was generated by an honest node and includes a transaction in which Alice pays Bob for the software download. Upon seeing this transaction included in the block chain, Bob concludes that Alice has paid him and allows Alice to download the software. Suppose the next random node that is selected in the next round happens to be controlled by Alice. Now since Alice gets to propose the next block, she could propose a block that ignores the block that contains the payment to Bob and instead contains a pointer to the previous block. Furthermore, in the block that she proposes, Alice includes a transaction that transfers the very coins that she was sending to Bob to a different address that she herself controls. This is a classic double-spend pattern. Since the two transactions spend the same coins, only one of them can be included in the block chain. If Alice succeeds in including the payment to her own address in the block chain, then the transaction in which she pays Bob is useless as it can never be included later in the block chain.

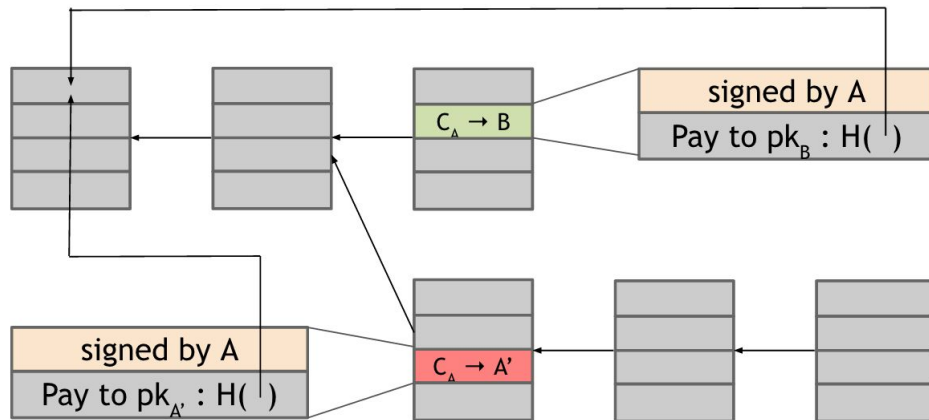


Figure 2.2 A double spend attempt. Alice creates two transactions: one in which she sends Bob Bitcoins, and a second in which she double spends those Bitcoins by sending them to a different address that she controls. As they spend the same Bitcoins, only one of these transactions can be included in the block chain. The arrows are pointers from one block to the previous block that it extends including a hash of that previous block within its own contents. C_A is used to denote a coin owned by Alice.

And how do we know if this double spend attempt is going to succeed or not? Well, that depends on which block will ultimately end up on the long-term consensus chain — the one with the Alice → Bob transaction or the one with the Alice → Alice transaction. What determines which block will be included? Honest nodes follow the policy of extending the longest valid branch, so which branch will they extend? There is no right answer! At this point, the two branches are the same length — they only differ in the last block and both of these blocks are valid. The node that chooses the next block then may decide to build upon either one of them, and this choice will largely determine whether or not the double-spend succeeds.

A subtle point: from a moral point of view, there is a clear difference between the block containing the transaction that pays Bob and the block containing the transaction in which Alice double spends those coins to her own address. But this distinction is only based on our knowledge of the story that Alice first paid Bob and then attempted to double spend. From a technological point of view, however, these two transactions are completely identical and both blocks are equally valid. The nodes that are looking at this really have no way to tell which is the morally legitimate transaction.

In practice, nodes often follow a heuristic of extending the block that they first heard about on the peer-to-peer network. But it's not a solid rule. And in any case, because of network latency, it could easily be that the block that a node first heard about is actually the one that was created second. So there is at least some chance that the next node that gets to propose a block will extend the block containing the double spend. Alice could further try to increase the likelihood of this happening by bribing the next node to do so. If the next node does build on the double-spend block for whatever reason, then this chain will now be longer than the one that includes the transaction to Bob. At this

point, the next honest node is much more likely to continue to build on this chain since it is longer. This process will continue, and it will become increasingly likely that the block containing the double-spend will be part of the long-term consensus chain. The block containing the transaction to Bob, on the other hand, gets completely ignored by the network, and this is now called an **orphan block**.

Let's now reconsider this whole situation from Bob-the-merchant's point of view. Understanding how Bob can protect himself from this double-spending attack is a key part of understanding Bitcoin security. When Alice broadcasts the transaction that represents her payment to Bob, Bob is listening on the network and hears about this transaction even before the next block is created. If Bob was even more foolhardy than we previously described, he can complete the checkout process on the website and allow Alice to download the software right at that moment. That's called a **zero-confirmation transaction**. This leads to an even more basic double spend attack than the one described before. Previously, for the double-spend attack to occur, we had to assume that a malicious actor controls the node that proposes the next block. But if Bob allows Alice to download the software before the transaction receives even a single confirmation on the block chain, then Alice can immediately broadcast a double-spend transaction, and an honest node may include it in the next block instead of the transaction that pays Bob.

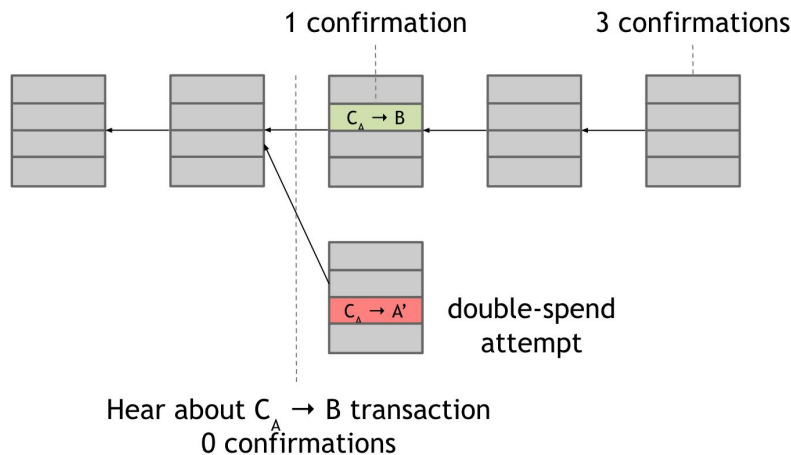


Figure 2.3 Bob the Merchant's view. This is what Alice's double-spend attempt looks like from Bob the merchant's viewpoint. In order to protect himself from this attack, Bob should wait until the transaction with which Alice pays him is included in the block chain and has several confirmations.

On the other hand, a cautious merchant would not release the software to Alice even after the transaction was included in one block, and would continue to wait. If Bob sees that Alice successfully launches a double-spend attack, he realizes that the block containing Alice's payment to him has been orphaned. He should abandon the transaction and not let Alice download the software. Instead, if it happens that despite the double-spend attempt, the next several nodes build on the block with the $Alice \rightarrow Bob$ transaction, then Bob gains confidence that this transaction will be on the long-term consensus chain.

In general, the more confirmations a transaction gets, the higher the probability that it is going to end up on the long-term consensus chain. Recall that honest nodes' behavior is always to extend the longest valid branch that they see. The chance that the shorter branch with the double spend will catch up to the longer branch becomes increasingly tiny as it grows longer than any other branch. This is especially true if only a minority of the nodes are malicious — for a shorter branch to catch up, several malicious nodes would have to be picked in close succession.

In fact, the double-spend probability decreases exponentially with the number of confirmations. So, if the transaction that you're interested in has received k confirmations, then the probability that a double-spend transaction will end up on the long-term consensus chain goes down exponentially as a function of k . The most common heuristic that's used in the Bitcoin ecosystem is to wait for six confirmations. There is nothing really special about the number six. It's just a good tradeoff between the amount of time you have to wait and your guarantee that the transaction you're interested in ends up on the consensus block chain.

To recap, protection against invalid transactions is entirely cryptographic. But it is enforced by consensus, which means that if a node does attempt to include a cryptographically invalid transaction, then the only reason that transaction won't end up in the long-term consensus chain is because a majority of the nodes are honest and won't include an invalid transaction in the block chain. On the other hand, protection against double-spending is purely by consensus. Cryptography has nothing to say about this, and two transactions that represent a double-spend attempt are both valid from a cryptographic perspective. But it's the consensus that determines which one will end up on the long-term consensus chain. And finally, you're never 100 percent sure that a transaction you're interested in is on the consensus branch. But, this exponential probability guarantee is rather good. After about six transactions, there's virtually no chance that you're going to go wrong.

2.4 Incentives and proof of work

In the previous section, we got a basic look at Bitcoin's consensus algorithm and a good intuition for why we believe that it's secure. But recall from the beginning of the chapter that Bitcoin's decentralization is partly a technical mechanism and partly clever incentive engineering. So far we've mostly looked at the technical mechanism. Now let's talk about the incentive engineering that happens in Bitcoin.

We asked you to take a leap of faith earlier in assuming that we're able to pick a random node and, perhaps more problematically, that at least 50 percent of the time, this process will pick an honest node. This assumption of honesty is particularly problematic if there are financial incentives for participants to subvert the process, in which case we can't really assume that a node will be honest. The question then becomes: can we give nodes an incentive for behaving honestly?

Consider again the double-spend attempt after one confirmation (Figure 2.3). Can we penalize,

somehow, the node that created the block with the double-spend transaction? Well, not really. As we mentioned earlier, it's hard to know which is the morally legitimate transaction. But even if we did, it's still hard to punish nodes since they don't have identities. So instead, let's flip the question around and ask, can we reward each of the nodes that created the blocks that did end up on the long-term consensus chain? Well, again, since those nodes don't reveal their real-world identities, we can't quite mail them cash to their home addresses. If only there were some sort of digital currency that we could use instead... you can probably see where this is going. We're going to use bitcoins to incentivize the nodes that created these blocks.

Let's pause for a moment. Everything that we've described so far is just an abstract algorithm for achieving distributed consensus and is not specific to the application. Now we're going to break out of that model, and we're going to use the fact that the application we're building through this distributed consensus process is in fact a currency. Specifically, we're going to incentivize nodes to behave honestly by paying them in units of this currency.

Block Reward. How is this done? There are two separate incentive mechanisms in Bitcoin. The first is the **block reward**. According to the rules of Bitcoin, the node that creates a block gets to include a special transaction in that block. This transaction is a coin-creation transaction, analogous to CreateCoins in ScroogeCoin, and the node can also choose the recipient address of this transaction. Of course that node will typically choose an address belonging to itself. You can think of this as a payment to the node in exchange for the service of creating a block on the consensus chain.

At the time of this writing, the value of the block reward is fixed at 25 Bitcoins. But it actually halves every 210,000 blocks. Based on the rate of block creation that we will see shortly, this means that the rate drops roughly every four years. We're now in the second period. For the first four years of Bitcoin's existence, the block reward was 50 bitcoins; now it's 25. And it's going to keep halving. This has some interesting consequences, which we will see shortly.

You may be wondering why the block reward incentivizes honest behavior. It may appear, based on what we've said so far, that this node gets the block reward regardless of whether it proposes a valid block or behaves maliciously. But this is not true! Think about it — how will this node “collect” its reward? That will only happen if the block in question ends up on the long-term consensus branch because just like every other transaction, the coin-creation transaction will only be accepted by other nodes if it ends up on the consensus chain. That's the key idea behind this incentive mechanism. It's a very subtle but very powerful trick. It incentivizes nodes to behave in whatever way they believe will get other nodes to extend their blocks. So if most of the network is following the longest valid branch rule, it incentivizes all nodes to continue to follow that rule. That's Bitcoin's first incentive mechanism.

We mentioned that every 210,000 blocks (or approximately four years), the block reward is cut in half. In Figure 2.4, the slope of this curve is going to keep halving. This is a geometric series, and you might know that it means that there is a finite sum. It works out to a total of 21 million bitcoins.

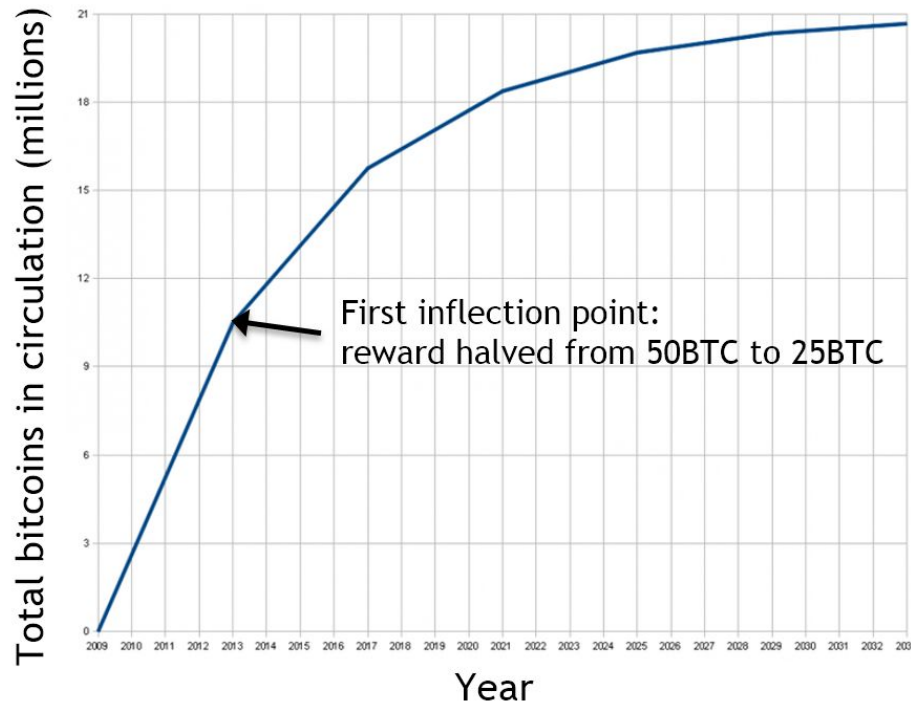


Figure 2.4 The block reward is cut in half every four years limiting the total supply of bitcoins to 21 million.

It is important to note that this is the only way in which new bitcoins are allowed to be created. There is no other coin generation mechanism, and that's why 21 million is a final and total number (as the rules stand now, at least) for how many bitcoins there can ever be. This new block creation reward is actually going to run out in 2140, as things stand now. Does that mean that the system will stop working in 2140 and become insecure because nodes no longer have the incentive to behave honestly? Not quite. The block reward is only the first of two incentive mechanisms in Bitcoin.

Transaction fees The second incentive mechanism is called the **transaction fee**. The creator of any transaction can choose to make the total value of the transaction outputs less than the total value of its inputs. Whoever creates the block that first puts that transaction into the block chain gets to collect the difference, which acts as a transaction fee. So if you're a node that's creating a block that contains, say, 200 transactions, then the sum of all those 200 transaction fees is paid to the address that you put into that block. The transaction fee is purely voluntary, but we expect, based on our understanding of the system, that as the block reward starts to run out, it will become more and more important, almost mandatory, for users to include transaction fees in order to get a reasonable quality of service. To a certain degree, this is already starting to happen now. But it is yet unclear precisely how the system will evolve; it really depends on a lot of game theory which hasn't been fully worked out yet. That's an interesting area of open research in Bitcoin.

There are still a few problems remaining with the consensus mechanism as we described it. The first

major one is the leap of faith that we asked you to take that somehow we can pick a random node. Second, we've created a new problem by giving nodes these incentives for participation. The system can become unstable as the incentives cause a free-for-all where everybody wants to run a Bitcoin node in the hope of capturing some of these rewards. And a third one is an even trickier version of this problem, which is that an adversary might create a large number of Sybil nodes to try and subvert the consensus process.

Mining and proof-of-work. It turns out that all of these problems are related, and all of them have the same solution, which is called **proof-of-work**. The key idea behind proof-of-work is that we approximate the selection of a random node by instead selecting nodes in proportion to a resource that we hope that nobody can monopolize. If, for example, that resource is computing power, then it's a proof-of-work system. Alternately, it could be in proportion to ownership of the currency, and that's called **proof-of-stake**. Although it's not used in Bitcoin, proof-of-stake is a legitimate alternate model and it's used in other cryptocurrencies. We'll see more about proof-of-stake and other proof-of-work variants in Chapter 8.

But back to proof-of-work. Let's try to get a better idea of what it means to select nodes in proportion to their computing power. Another way of understanding this is that we're allowing nodes to compete with each other by using their computing power, and that will result in nodes automatically being picked in that proportion. Yet another view of proof-of-work is that we're making it moderately hard to create new identities. It's sort of a tax on identity creation and therefore on the Sybil attack. This might all appear a bit vague, so let's go ahead and look at the details of the proof-of-work system that's used in Bitcoin, which should make things a lot clearer.

Bitcoin achieves proof-of-work using **hash puzzles**. In order to create a block, the node that proposes that block is required to find a number, or **nonce**, such that when you concatenate the nonce, the previous hash, and the list of transactions that comprise that block and take the hash of this whole string, then that hash output should be a number that falls into a target space that is quite small in relation to the much larger output space of that hash function. We can define such a target space as any value falling below a certain target value. In this case, the nonce will have to satisfy the following inequality:

$$H(\text{nonce} \parallel \text{prev_hash} \parallel \text{tx} \parallel \text{tx} \parallel \dots \parallel \text{tx}) < \text{target}$$

As we saw earlier, normally a block contains a series of transactions that a node is proposing. In addition, a block also contains a hash pointer to the previous block¹. In addition, we're now requiring that a block also contain a nonce. The idea is that we want to make it moderately difficult to find a nonce that satisfies this required property, which is that hashing the whole block together, including that nonce, is going to result in a particular type of output. If the hash function satisfies the

¹ We are using the term hash pointer loosely. The pointer is just a string in this context as it need not tell us where to find this block. We will find the block by asking other peers on the network for it. The important part is the hash that both acts as an ID when requesting other peers for the block and lets us validate the block once we have obtained it.

puzzle-friendliness property from Chapter 1, then the only way to succeed in solving this hash puzzle is to just try enough nonces one by one until you get lucky. So specifically, if this target space were just one percent of the overall output space, you would have to try about 100 nonces before you got lucky. In reality, the size of this target space is not nearly as high as one percent of the output space. It's much, much smaller than that as we will see shortly.

This notion of hash puzzles and proof of work completely does away with the requirement to magically pick a random node. Instead, nodes are simply independently competing to solve these hash puzzles all the time. Once in a while, one of them will get lucky and will find a random nonce that satisfies this property. That lucky node then gets to propose the next block. That's how the system is completely decentralized. There is nobody deciding which node it is that gets to propose the next block.

Difficult to compute. There are three important properties of hash puzzles. The first is that they need to be quite difficult to compute. We said moderately difficult, but you'll see why this actually varies with time. As of the end of 2014, the difficulty level is about 10^{20} hashes per block. In other words the size of the target space is only $1/10^{20}$ of the size of the output space of the hash function. This is a lot of computation — it's out of the realm of possibility for a commodity laptop, for example. Because of this, only some nodes even bother to compete in this block creation process. This process of repeatedly trying and solving these hash puzzles is known as **Bitcoin mining**, and we call the participating nodes **miners**. Even though technically anybody can be a miner, there's been a lot of concentration of power in the mining ecosystem due to the high cost of mining.

Parameterizable cost. The second property is that we want the cost to be parameterizable, not a fixed cost for all time. The way that's accomplished is that all the nodes in the Bitcoin peer-to-peer network will automatically recalculate the target, that is the size of the target space as a fraction of the output space, every 2016 blocks. They recalculate the target in such a way that the average time between successive blocks produced in the Bitcoin network is about 10 minutes. With a 10-minute average time between blocks, 2016 blocks works out to two weeks. In other words, the recalculation of the target happens roughly every two weeks.

Let's think about what this means. If you're a miner, and you've invested a certain fixed amount of hardware into Bitcoin mining, but the overall mining ecosystem is growing, more miners are coming in, or they're deploying faster and faster hardware, that means that over a two week period, slightly more blocks are going to be found than expected. So nodes will automatically readjust the target, and the amount of work that you have to do to be able to find a block is going to increase. So if you put in a fixed amount of hardware investment, the rate at which you find blocks is actually dependent upon what other miners are doing. There's a very nice formula to capture this, which is that the probability that any given miner, Alice, is going to win the next block is equivalent to the fraction of global hash power that she controls. This means that if Alice has mining hardware that's about 0.1 percent of total hash power, she will find roughly one in every 1,000 blocks.

What is the purpose of this readjustment? Why do we want to maintain this 10-minute invariant? The

reason is quite simple. If blocks were to come very close together, then there would be a lot of inefficiency, and we would lose the optimization benefits of being able to put a lot of transactions in a single block. There is nothing magical about the number 10, and if you went down from 10 minutes to 5 minutes, it would probably be just fine. There's been a lot of discussion about the ideal block latency that altcoins, or alternative cryptocurrencies, should have. But despite some disagreements about the ideal latency, everybody agrees that it should be a fixed amount. It cannot be allowed to go down without limit. That's why we have the automatic target recalculation feature.

The way that this cost function and proof of work is set up allows us to reformulate our security assumption. Here's where we finally depart from the last leap of faith that we asked you to take earlier. Instead of saying that somehow the majority of nodes are honest in a context where nodes don't even have identities and not being clear about what that means, we can now state crisply, that a lot of attacks on Bitcoin are infeasible if the majority of miners, weighted by hash power, are following the protocol — or, are honest. This is true because if a majority of miners, weighted by hash power, are honest, the competition for proposing the next block will automatically ensure that there is at least a 50 percent chance that the next block to be proposed at any point is coming from an honest node.

Sidebar. in the research fields of distributed systems and computer security, it is common to assume that some percentage of nodes are honest and to show that the system works as intended even if the other nodes behave arbitrarily. That's basically the approach we've taken here, except that we weight nodes by hash power in computing the majority. The original Bitcoin whitepaper contains this type of analysis as well.

But the field of game theory provides an entirely different, and arguably more sophisticated and realistic way to determine how a system will behave. In this view, we don't split nodes into honest and malicious. Instead, we assume that *every* node acts according to its incentives. Each node picks a (randomized) strategy to maximize its payoff, taking into account other nodes' potential strategies. If the protocol and incentives are designed well, then most nodes will follow the rules most of the time. "Honest" behavior is just one strategy of many, and we attach no particular moral salience to it.

In the game theoretic view, the big question is whether the default miner behavior is a "Nash equilibrium," that is, whether it represents a stable situation in which no miner can realize a higher payoff by deviating from honest behavior. This question is still contentious and an active area of research.

Solving hash puzzles is probabilistic because nobody can predict which nonce is going to result in solving the hash puzzle. The only way to do it is to try nonces one by one and hope that one succeeds. Mathematically, this process is called **Bernoulli trials**. A Bernoulli trial is an experiment with two

possible outcomes, and the probability of each outcome occurring is fixed between successive trials. Here, the two outcomes are whether or not the hash falls in the target, and assuming the hash functions behaves like a random function, the probability of those outcomes is fixed. Typically, nodes try so many nonces that Bernoulli trials, a discrete probability process, can be well approximated by a continuous probability process called a **Poisson process**, a process in which events occur independently at a constant average rate. The end result of all of that is that the probability density function that shows the relative likelihood of the time until the next block is found looks like Figure 2.5.

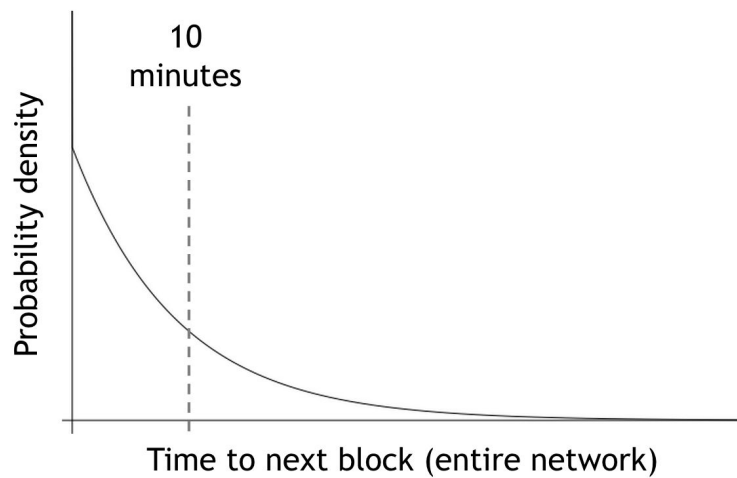


Figure 2.5 Probability density function of the time until the next block is found.

This is known as an exponential distribution. There is some small probability that if a block has been found now, the next block is going to be found very soon, say within a few seconds or a minute. And there is also some small probability that it will take a long time, say an hour, to find the next block. But overall, the network automatically adjusts the difficulty so that the inter-block time is maintained at an average, long term, of 10 minutes. Notice that Figure 2.5 shows how frequently blocks are going to be created by the entire network not caring about which miner actually finds the block.

If you're a miner, you're probably interested in how long it will take you to find a block. What does this probability density function look like? It's going to have the same shape, but it's just going to have a different scale on the x-axis. Again, it can be represented by a nice equation.

For a specific miner:

$$\text{mean time to next block} = \frac{10 \text{ minutes}}{\text{fraction of hash power}}$$

If you have 0.1 percent of the total network hash power, this equation tells us that you're going to find blocks once every 10,000 minutes, which is just about a week. Not only is your mean time between blocks going to be very high, but the variance of the time between blocks found by you is also going to be very high. This has some important consequences that we're going to look at in chapter 5.

Trivial to verify. Let's now turn to the third important property of this proof of work function, which is that it is trivial to verify that a node has computed proof of work correctly. Even if it takes a node, on average, 10^{20} tries to find a nonce that makes the block hash fall below the target, that nonce must be published as part of the block. It is thus trivial for any other node to look at the block contents, hash them all together, and verify that the output is less than the target. This is quite an important property because, once again, it allows us to get rid of centralization. We don't need any centralized authority verifying that miners are doing their job correctly. Any node or any miner can instantly verify that a block found by another miner satisfies this proof-of-work property.

2.5 Putting it all together

Cost of mining. Let's now look at mining economics. We mentioned it's quite expensive to operate as a miner. At the current difficulty level, finding a single block takes computing about 10^{20} hashes and the block reward is about 25 Bitcoins, which is a sizable amount of money at the current exchange rate. These numbers allow for an easy calculation of whether it's profitable for one to mine, and we can capture this decision with a simple statement:

If
 mining reward > mining cost
then miner profits
where
 mining reward = block reward + tx fees
 mining cost = hardware cost + operating costs (electricity, cooling, etc.)

Fundamentally, the mining reward that the miner gets is in terms of the block reward and transaction fees. The miner asks himself how it compares to the total expenditure, which is the hardware and electricity cost.

But there are some complications to this simple equation. The first is that, as you may have noticed, the hardware cost is a fixed cost whereas the electricity cost is a variable cost that is incurred over time. Another complication is that the reward that miners get depends upon the rate at which they find blocks, which depends on not just the power of their hardware, but on the ratio of their hash rate to the total global hash rate. A third complication is that the costs that the miner incurs are typically denominated in dollars or some other traditional currency, but their reward is denominated in bitcoin. So this equation has a hidden dependence on Bitcoin's exchange rate at any given time. And finally, so far we've assumed that the miner is interested in honestly following the protocol. But the miner might choose to use some other mining strategy instead of always attempting to extend the longest valid branch. So this equation doesn't capture all the nuances of the different strategies that the miner can employ. Actually analyzing whether it makes sense to mine is a complicated game

theory problem that's not easily answered.

At this point, we've obtained a pretty good understanding of how a Bitcoin achieves decentralization. We will now recap the high level points and put it all together in order to get an even better understanding.

Let's start from identities. As we've learned, there are no real-world identities required to participate in the Bitcoin protocol. Any user can create a pseudonymous key pair at any moment, any number of them. When Alice wants to pay Bob in bitcoins, the Bitcoin protocol does not specify how Alice learns Bob's address. Given these pseudonymous key pairs as identities, transactions are basically messages that are broadcast to the Bitcoin peer-to-peer network that are instructions to transfer coins from one address to another. Bitcoins are just transaction outputs, and we will discuss this in much more detail in the next chapter.

Sidebar. Bitcoin doesn't have fixed denominations like US dollars, and in particular, there is no special designation of "1 bitcoin." Bitcoins are just transaction outputs, and in the current rules, they can have an arbitrary value with 8 decimal places of precision. The smallest possible value is 0.00000001 BTC (bitcoins), which is called 1 **Satoshi**.

The goal of the Bitcoin peer-to-peer network is to propagate all new transactions and new blocks to all the Bitcoin peer nodes. But the network is highly imperfect, and does a best-effort attempt to relay this information. The security of the system doesn't come from the perfection of the peer-to-peer network. Instead, the security comes from the block chain and the consensus protocol that we devoted much of this chapter to studying.

When we say that a transaction is included in the block chain, what we really mean is that the transaction has achieved numerous confirmations. There's no fixed number to how many confirmations are necessary before we are sufficiently convinced of its inclusion, but six is a commonly-used heuristic. The more confirmations a transaction has received, the more certain you can be that this transaction is part of the consensus chain. There will often be orphan blocks, or blocks that don't make it into the consensus chain. There are a variety of reasons that could lead to a block being orphaned. The block may contain an invalid transaction, or a double-spend attempt. It could also just be a result of network latency. That is, two miners may simply end up finding new blocks within just a few seconds of each other. So both of these blocks were broadcast nearly simultaneously onto the network, and one of them will inevitably be orphaned.

Finally, we looked at hash puzzles and mining. Miners are special types of nodes that decide to compete in this game of creating new blocks. They're rewarded for their effort in terms of both newly minted bitcoins (the new-block reward) and existing bitcoins (transaction fees), provided that other miners build upon their blocks. A subtle but crucial point: say that Alice and Bob are two different miners, and Alice has 100 times as much computing power as Bob. This does not mean that Alice will always win the race against Bob to find the next block. Instead, Alice and Bob have a probability ratio

of finding the next block, in the proportion 100 to 1. In the long term, Bob will find, on average, one percent of the number of blocks that Alice finds.

We expect that miners will typically be somewhere close to the economic equilibrium in the sense that the expenditure that they incur in terms of hardware and electricity will be roughly equal to the rewards that they obtain. The reason is that if a miner is consistently making a loss, she will probably stop mining. On the other hand, and if mining is very profitable given typical hardware and electricity costs, then more mining hardware would enter the network. The increased hash rate would lead to an increase in the difficulty, and each miner's expected reward would drop.

This notion of distributed consensus permeates Bitcoin quite deeply. In a traditional currency, consensus does come into play to a certain limited extent. Specifically, there is a consensus process that determines the exchange rate of the currency. That is certainly true in Bitcoin as well; We need consensus around the value of Bitcoin. But in Bitcoin, additionally, we need consensus on the state of the ledger, which is what the block chain accomplishes. In other words, even the accounting of how many bitcoins you own is subject to consensus. When we say that Alice owns a certain amount or number of bitcoins, what we actually mean is that the Bitcoin peer-to-peer network, as recorded in the block chain, considers the sum total of all Alice's addresses to own that number of bitcoins. That is ultimate nature of truth in Bitcoin: ownership of bitcoins is nothing more than other nodes agreeing that a given party owns those bitcoins.

Finally, we need consensus about the rules of the system because occasionally, the rules of the system have to change. There are two types of changes to the rules of Bitcoin, known respectively as **soft forks** and **hard forks**. We're going to defer this discussion of the differences to later chapters in which we will discuss them in detail.

Getting a cryptocurrency off the ground. Another subtle concept is that of **bootstrapping**. There is a tricky interplay between three different ideas in Bitcoin: the security of the block chain, the health of the mining ecosystem, and the value of the currency. We obviously want the block chain to be secure for Bitcoin to be a viable currency. For the block chain to be secure, an adversary must not be able to overwhelm the consensus process. This in turn means that an adversary cannot create a lot of mining nodes and take over 50 percent or more of the new block creation.

But when will that be true? A prerequisite is having a healthy mining ecosystem made up of largely honest, protocol-following nodes. But what's a prerequisite for that — when can we be sure that a lot of miners will put a lot of computing power into participating in this hash puzzle solving competition? Well, they're only going to do that if the exchange rate of Bitcoin is pretty high because the rewards that they receive are denominated in Bitcoins whereas their expenditure is in dollars. So the more the value of the currency goes up, the more incentivized these miners are going to be.

But what ensures a high and stable value of the currency? That can only happen if users in general have trust in the security of the block chain. If they believe that the network could be overwhelmed at any moment by an attacker, then Bitcoin is not going to have a lot of value as a currency. So you have

this interlocking interdependence between the security of the block chain, a healthy mining ecosystem and the exchange rate.

Because of the cyclical nature of this three-way dependence, the existence of each of these is predicated on the existence of the others. When Bitcoin was first created, none of these three existed. There were no miners other than Nakamoto himself running the mining software. Bitcoin didn't have a lot of value as a currency. And the block chain was, in fact, insecure because there was not a lot of mining going on and anybody could have easily overwhelmed this process.

There's no simple explanation for how Bitcoin went from not having any of these properties to having all three of them. Media attention was part of the story — the more people hear about Bitcoin, the more they're going to get interested in mining. And the more they get interested in mining, the more confidence people will have in the security of the block chain because there's now more mining activity going on, and so forth. Incidentally, every new Altcoin that wants to succeed also has to somehow solve this problem of pulling itself up by its bootstraps.

51-percent attack. Finally, let's consider what would happen if consensus failed and there was in fact a **51-percent attacker** who controls 51 percent or more of the mining power in the Bitcoin network. We'll consider a variety of possible attacks and see which ones can actually be carried out by such an attacker.

First of all, can this attacker steal coins from an existing address? As you may have guessed, the answer is no, because stealing from an existing address is not possible unless you subvert the cryptography. It's not enough to subvert the consensus process. This is not completely obvious. Let's say the 51 percent attacker creates an invalid block that contains an invalid transaction that represents stealing Bitcoins from an existing address that the attacker doesn't control and transferring them to his own address. The attacker can pretend that it's a valid transaction and keep building upon this block. The attacker can even succeed in making that the longest branch. But the other honest nodes are simply not going to accept this block with an invalid transaction and are going to keep mining based on the last valid block that they found in the network. So what will happen is that there will be what we call a fork in the chain.

Now imagine this from the point of view of the attacker trying to spend these invalid coins, and send them to some merchant Bob as payment for some goods or service. Bob is presumably running a Bitcoin node himself, and it will be an honest node. Bob's node will reject that branch as invalid because it contains an invalid transaction. It's invalid because the signatures didn't check out. So Bob's node will simply ignore the longest branch because it's an invalid branch. And because of that, subverting consensus is not enough. You have to subvert cryptography to steal bitcoins. So we conclude that this attack is not possible for a 51 percent attacker.

We should note that all this is only a thought experiment. If there were, in fact, actual signs of a 51 percent attack, what will probably happen is that the developers will notice this and react to it. They will update the Bitcoin software, and we might expect that the rules of the system, including the

peer-to-peer network, might change in some form to make it more difficult for this attack to succeed. But we can't quite predict that. So we're working in a simplified model where a 51 percent attack happens, but other than that, there are no changes or tweaks to the rules of the system.

Let's consider another attack. Can the 51-percent attacker suppress some transactions? Let's say there is some user, Carol, whom the attacker really doesn't like. The attacker knows some of Carol's addresses, and wants to make sure that no coins belonging to any of those addresses can possibly be spent. Is that possible? Since he controls the consensus process of the block chain, the attacker can simply refuse to create any new blocks that contain transactions from one of Carol's addresses. The attacker can further refuse to build upon blocks that contain such transactions. However, he can't prevent these transactions from being broadcast to the peer-to-peer network because the network doesn't depend on the block chain, or on consensus, and we're assuming that the attacker doesn't fully control the network. The attacker cannot stop the transactions from reaching the majority of nodes, so even if the attack succeeds, it will at least be apparent that the attack is happening.

Can the attacker change the block reward? That is, can the attacker start pretending that the block reward is, instead of 25 Bitcoins, say 100 Bitcoins? This is a change to the rules of the system, and because the attacker doesn't control the copies of the Bitcoin software that all of the honest nodes are running, this is also not possible. This is similar to the reason why the attacker cannot include invalid transactions. Other nodes will simply not recognize the increase in the block reward, and the attacker will thus be unable to spend them.

Finally, can the attacker somehow destroy confidence in Bitcoin? Well, let's imagine what would happen. If there were a variety of double-spend attempts, situations in which nodes did not extend the longest valid branch, and other attempted attacks, then people are going to likely decide that Bitcoin is no longer acting as a decentralized ledger that they can trust. People will lose confidence in the currency, and we might expect that the exchange rate of Bitcoin will plummet. In fact, if it is known that there is a party that controls 51 percent of the hash power, then it's possible that people will lose confidence in Bitcoin even if the attacker is not necessarily trying to launch any attacks. So it is not only possible, but in fact likely, that a 51 percent attacker of any sort will destroy confidence in the currency. Indeed, this is the main practical threat if a 51 percent attack were ever to materialize. Considering the amount of expenditure that the adversary would have to put into attacking Bitcoin and achieving a 51 percent majority, none of the other attacks that we described really make sense from a financial point of view.

Hopefully, at this point you've obtained a really good understanding of how decentralization is achieved in Bitcoin. You should have a good command on how identities work in Bitcoin, how transactions are propagated and validated, the role of the peer-to-peer network in Bitcoin, how the block chain is used to achieve consensus, and how hash puzzles and mining work. These concepts provide a solid foundation and a good launching point for understanding a lot of the more subtle details and nuances of Bitcoin, which we're going to see in the coming chapters.

Further reading

The Bitcoin whitepaper:

Nakamoto, Satoshi. [Bitcoin: A peer-to-peer electronic cash system](#). (2008)

The original application of proof-of-work:

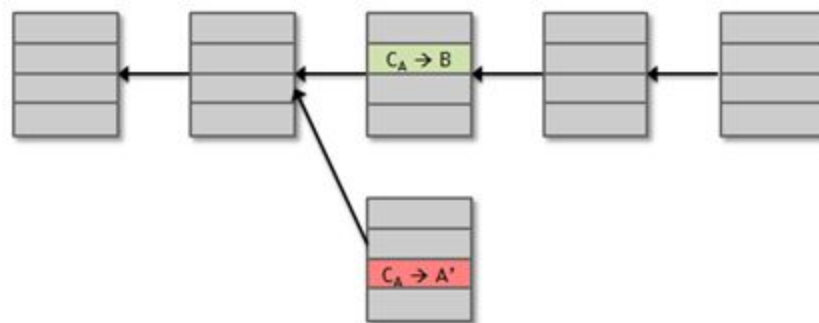
Back, Adam. [Hashcash-a denial of service counter-measure](#). (2002)

The Paxos algorithm for consensus:

Lamport, Leslie. [Paxos made simple](#). ACM Sigact News 32.4 (2001): 18-25.

Exercises

1. Why do miners run “full nodes” that keep track of the entire block chain² whereas Bob the merchant can get away with a “lite node” that implements “simplified payment verification,” needing to examine only the last few blocks?
2. If a malicious ISP completely controls a user’s connections, can it launch a double-spend attack against the user? How much computational effort would this take?
3. Consider Bob the merchant deciding whether or not to accept the $C_A \rightarrow B$ transaction. What Bob is really interested in is whether or not the other chain will catch up. Why, then, does he simply check how many confirmations $C_A \rightarrow B$ has received, instead of computing the difference in length between the two chains?



² This only applies to “solo” miners who’re not part of a mining pool, but we haven’t discussed that yet.

4. Even when all nodes are honest, blocks will occasionally get orphaned: if two miners Minnie and Mynie discover blocks nearly simultaneously, neither will have time to hear about the other's block before broadcasting hers.

4a. What determines whose block will end up on the consensus branch?

4b. What factors affect the rate of orphan blocks? Can you derive a formula for the rate based on these parameters?

4c. Try to empirically measure this rate on the Bitcoin network.

4d. If Mynie hears about Minnie's block just before she's about to discover hers, does that mean she wasted her effort?

4e. Do all miners have their blocks orphaned at the same rate, or are some miners affected disproportionately?

5a. How can a miner establish an identity in a way that's hard to fake? (i.e., anyone can tell which blocks were mined by her.)

5b. If a miner misbehaves, can other miners "boycott" her by refusing to build on her blocks on an ongoing basis?

6a. Assuming that the total hash power of the network stays constant, what is the probability that a block will be found in the next 10 minutes?

6b. Suppose Bob the merchant wants to have a policy that orders will ship within x minutes after receipt of payment. What value of x should Bob choose so that with 99% confidence 6 blocks will be found within x minutes?